
Engenharia de Software Experimental apoiada por ontologia

Lilian Passos Scatalon

Engenharia de Software Experimental apoiada por ontologia

Lilian Passos Scatalon

Orientador: *Prof. Dr. Rogério Eduardo Garcia*

Monografia referente ao exame de qualificação, apresentada ao Programa de Pós-Graduação em Ciência da Computação (PPGCC-UNESP) como parte dos requisitos para a obtenção do título de mestre em Ciência da Computação.

Índice

Lista de Figuras	iii
Lista de Tabelas	iv
Resumo	v
1 Introdução	1
2 Experimentação em Engenharia de Software	4
2.1 Experimento controlado	4
2.2 Processo de Experimentação	6
2.2.1 Definição	7
2.2.2 Planejamento	8
2.2.3 Operação	9
2.2.4 Análise e interpretação	9
2.2.5 Empacotamento	10
2.3 Diretrizes para reportar experimentos controlados	10
2.4 Replicações e Transferência de Conhecimento	13
2.5 A <i>ExperOntology</i>	14
2.6 Considerações finais	16
3 Matching de esquemas/ontologias	17
3.1 Definições gerais e operação de <i>matching</i>	18
3.2 Processo de <i>matching</i>	21
3.2.1 Execução	22
3.2.2 Combinação de resultados	22
3.2.3 Seleção de correspondências	24
3.3 Considerações finais	24
4 Técnicas de <i>matching</i>	25
4.1 Técnica: uma função de similaridade	25
4.2 Técnicas baseadas em termos	26
4.2.1 <i>Strings</i>	26

4.2.2	Palavras	29
4.3	Técnicas baseadas na estrutura	31
4.3.1	Estrutura interna de uma entidade	31
4.3.2	Estrutura relacional entre entidades	32
4.4	Técnicas baseadas em instâncias	33
4.5	Técnicas baseadas em um modelo semântico	34
4.6	Considerações finais	37
5	Projeto de Pesquisa	39
5.1	Objetivos	39
5.2	Justificativa	40
5.3	Metodologia	41
5.4	Atividades e Cronograma	43
5.5	Resultados Parciais	44
6	Considerações finais	48
	Referências Bibliográficas	53

Lista de Figuras

2.1	Relação causal avaliada em um experimento controlado (Wohlin et al., 2000)	5
2.2	Variáveis independentes e dependentes (Wohlin et al., 2000)	6
2.3	Processo de experimentação (Wohlin et al. (2000); Garcia (2006))	7
2.4	Processo de experimentação - fase de empacotamento (Wohlin et al. (2000); Amaral e Travassos (2003))	11
2.5	Processo de experimentação (Mendonça et al., 2008)	14
2.6	Conceitos de um experimento controlado (Garcia et al., 2008)	15
2.7	Conceitos de um pacote de laboratório (Garcia et al., 2008)	16
3.1	Conciliando dois esquemas diferentes (Halevy, 2005)	19
3.2	Operação de <i>matching</i> (Euzenat e Shvaiko, 2007)	20
3.3	Processo de <i>matching</i> de esquemas (Rahm, 2011)	21
3.4	Composição sequencial de <i>matchers</i> (Euzenat e Shvaiko, 2007)	22
3.5	Composição paralela de <i>matchers</i> (Euzenat e Shvaiko, 2007)	23
3.6	Matrizes de dissimilaridade (Euzenat e Shvaiko, 2007)	23
3.7	Matriz resultante da agregação (Euzenat e Shvaiko, 2007)	23
4.1	<i>Matching</i> semântico com o auxílio de alinhamentos âncora (Aleksovski et al., 2006) .	35
4.2	Classificação das técnicas de <i>matching</i> de acordo com o tipo de entrada (adaptada de Euzenat e Shvaiko (2007))	38
5.1	<i>Workflow</i> para empacotar experimentos controlados (Scatalon et al., 2011a)	40
5.2	Processo de descoberta de conhecimento (Maimon e Rokach, 2010)	42
5.3	Etapa de pré-processamento do processo de <i>matching</i>	45
5.4	Processo de KDD adaptado	45
5.5	Extração de elementos do arquivo PDF	46

Lista de Tabelas

2.1	Exemplo de atribuição de participantes a tratamentos Wohlin et al. (2000)	9
2.2	Exemplo de um projeto com dois fatores Wohlin et al. (2000)	9
2.3	Visão geral de testes paramétricos/não-paramétricos para diferentes projetos Wohlin et al. (2000)	10
2.4	Visão geral das propostas de diretrizes de empacotamento (Jedlitschka et al., 2008) .	12
4.1	Exemplo de vetores com as frequências dos termos de cada documento	28
4.2	Parte de uma tabela de compatibilidade de tipos de dados (Euzenat e Shvaiko, 2007)	31
4.3	Uma tabela de compatibilidade de cardinalidade (Euzenat e Shvaiko, 2007)	32
4.4	Exemplo de estrutura interna de entidades a serem casadas (Rahm e Bernstein, 2001)	32
4.5	Técnicas básicas de <i>matching</i>	38
5.1	Cronograma de atividades do projeto de mestrado	43
5.2	Bibliotecas e ferramentas de extração de texto de arquivos PDF (Pitale e Sharma, 2011)	46

Resumo

Engenharia de Software Experimental (ESE) visa a avaliar e medir o desempenho de modelos e técnicas relacionados ao desenvolvimento de software para, assim, construir um corpo de conhecimento que apóie tomadas de decisão. Para construí-lo, é necessária a condução de experimentos controlados e suas replicações a fim de generalizar os resultados obtidos. O conhecimento gerado por um estudo é registrado no chamado pacote de laboratório, o qual deve ser revisado por um eventual grupo de pesquisa com a intenção de replicá-lo. E para viabilizar a integração dos resultados, os pacotes de laboratório devem permitir a sua combinação em um corpo comum. Entretanto, tem sido observado que publicações de experimentos contêm diferentes conjuntos de informações, de acordo com o que parece ser mais relevante para cada pesquisador. Por outra perspectiva, pesquisadores enfrentam dificuldades ao revisar os pacotes de laboratório disponíveis, o que causa a necessidade de padronizar e também melhorar o entendimento do conteúdo de pacotes de laboratório. Nesse contexto, considerando que compartilhamento e integração de conhecimento são aplicações recorrentes de ontologias, este trabalho propõe um workflow para empacotar estudos baseado na ExperOntology, uma ontologia do domínio de experimentos controlados. A execução iterativa do workflow também evolui as definições da ontologia, a fim de que seja um padrão semântico para acomodar diferentes conjuntos de informações. Desse modo, esta abordagem promove um melhor entendimento de pacotes de laboratório e também lida com a questão da heterogeneidade, facilitando, portanto, a transferência de conhecimento entre grupos de pesquisa.

Introdução

Para atingir seus objetivos, equipes de desenvolvimento precisam entender e escolher modelos e técnicas apropriados para apoiar seus projetos. Nessa perspectiva, estudos experimentais vêm sendo realizados, contribuindo para a formação de uma base de conhecimento sobre Engenharia de Software (ES) que pode ajudar nas tomadas de decisão ao longo do desenvolvimento.

Tais estudos avaliam a aplicação de métodos, técnicas e ferramentas em atividades de desenvolvimento de software. Elabora-se um modelo similar a profissionais construindo um software e os participantes, que representam os engenheiros de software, aplicam a tecnologia sob investigação. Os dados produzidos pela execução permitem que o desempenho seja avaliado, medido e comparado, considerando as condições sob as quais o experimento foi executado. Para possibilitar o posterior acesso a todo o conhecimento gerado pelo estudo, todos os procedimentos, resultados e conclusões são armazenados no chamado pacote de laboratório.

A partir dos resultados de estudos experimentais é construído um corpo de conhecimento, que além de prover um alicerce científico para a disciplina de ES, favorece a sua respectiva aplicação no mercado, apoiando em tomadas de decisão. Entretanto, é imprescindível que os resultados sejam reproduzíveis, que haja replicações do estudo para fundamentar suas conclusões. [Brooks et al. \(2008\)](#) afirmam que altos níveis de confiança são alcançados apenas quando outros pesquisadores, além dos pertencentes ao grupo de pesquisa do estudo original, conseguem replicar um experimento com sucesso.

Realizar uma replicação, que não seja de maneira completamente independente, requer o acesso às informações do pacote de laboratório. Revisando como o estudo foi projetado, conduzido e analisado, replicadores podem compreender como o experimento original foi feito, e até perceber o que deve ser mudado para melhor modelar o fenômeno sob estudo. Entretanto, embora disponíveis para revisão, foram apontados problemas na compreensão de pacotes de laboratório por parte dos pesquisadores ([Shull et al., 2002](#)). A falta de entendimento compartilhado do conteúdo dos pacotes levam a dificuldades na transferência de conhecimento entre grupos de pesquisa. Para lidar com essas dificuldades, [Mendonça et al. \(2008\)](#) propuseram o arcabouço FIRE (*Framework for Improving the Replication of*

Experiments), composto de atividades que sugerem, entre outras práticas, a padronização dos pacotes e o compartilhamento do conhecimento gerado por cada experimento.

Compartilhar conhecimento é uma aplicação recorrente de ontologias, e, portanto, elas podem ser aplicadas na transferência de conhecimento em Engenharia de Software Experimental. Ontologias vêm sendo amplamente utilizadas em ES com a finalidade de modelar conceitos de um domínio e compartilhar de conhecimento (Kitchenham et al. (1999); Falbo et al. (2005); Barbosa et al. (2006); Biolchini et al. (2007)). Com esses mesmos objetivos, Garcia et al. (2008) apresentaram uma ontologia para o domínio de experimentos controlados, a *EXPEROntology*, que ajuda a esclarecer conceitos e relações relativos a experimentos controlados.

Ontologias provêm o entendimento compartilhado da informação que modelam (Noy e McGuinness, 2001). Assim, a principal finalidade da *EXPEROntology* é incorporar esse benefício à transferência de conhecimento embutido nos pacotes de laboratório, o que é essencial para efetivamente transferi-lo. Portanto, para facilitar o entendimento do conteúdo dos pacotes de laboratório, a *EXPEROntology* pode ser usada como um padrão semântico das informações que um pacote deve conter. Entretanto, não basta o estabelecimento de um padrão. Vários experimentos foram conduzidos e seus resultados publicados, o que contribui para a formação de um corpo de conhecimento em ES. Entretanto, cada pesquisador publica as informações que considera mais importantes no relatório do experimento (Jedlitschka et al., 2008). Portanto, é preciso prover meios para que os experimentos já realizados também sejam integrados em um corpo comum. O padrão deve facilitar o entendimento das informações e também considerar a prática de empacotamento atual, uma vez que pacotes de laboratório apresentam diferentes conjuntos de informação.

A fim de que os estudos realizados por grupos distintos não se tornem isolados, eles devem ser integrados a um corpo comum de conhecimento (Mendonça et al., 2008). Jedlitschka et al. (2008) apontam que a heterogeneidade das maneiras de se reportar os pacotes é um grande problema para a integração dos resultados dos estudos. De fato, os pacotes de laboratório encontrados pela literatura contêm conjuntos de informações diferentes ou níveis de detalhes diferentes.

Com uma análise dos problemas apresentados (dificuldades na transferência de conhecimento e heterogeneidade da estrutura de informação dos estudos publicados), é possível adotar a mesma abordagem para contorná-los. O empacotamento das informações sobre um estudo pode incorporar a ontologia como um padrão (Scatalon et al., 2011b) para lidar com as complicações na integração e transferência de conhecimento dos experimentos. E, considerando a necessidade de uma padronização que esteja aliada a resolver a heterogeneidade dos experimentos publicados (Carver, 2010), a *EXPEROntology* deve tratar as diferentes maneiras nas quais a execução de um experimento é registrada. Isso sugere que a ontologia inicial deve evoluir, incorporando novos conceitos.

Nesse contexto, é proposto um *workflow* para empacotar experimentos controlados usando a *EXPEROntology*, com a abordagem evolutiva de adicionar conceitos que falem na ontologia para instanciar satisfatoriamente diferentes pacotes de laboratório (Scatalon et al., 2011a). Assim, a ontologia pode ser vista como um padrão que promove o entendimento compartilhado da estrutura das informações do experimento. O objetivo é investigar o estabelecimento de um padrão para o pacote de laboratório, considerando durante a padronização a análise dos formatos de pacotes de estudos já

publicados. Conforme os pacotes de laboratório são instanciados de acordo com essa abordagem, a ontologia deve se aproximar de um padrão semântico que permite acomodar variações de pacotes de laboratório, e, portanto, a evolução da *EXPEROntology* também aprimora a sua aplicação em empacotar experimentos.

Organização do documento

Experimentação em Engenharia de Software

Segundo [Wohlin et al. \(2000\)](#) os seguintes requisitos são necessários para obter êxito no desenvolvimento de software: compreensão do processo e produto de software; definição de processo de desenvolvimento e qualidade do produto; avaliação de sucessos e falhas; realimentação de informação para controle do projeto; aprendizagem com a experiência; empacotamento e reutilização da experiência relevante. Estudos experimentais são ferramentas para obter os dados necessários para apoiar tais atividades.

Os estudos experimentais, de uma maneira geral, permitem construir um corpo de conhecimento válido em ES, que não seja apenas baseado em fontes não confiáveis, como intuição, especulações ou conhecimento convencional ([Travassos et al., 2002](#)). De acordo com [Basili et al. \(1999\)](#), o progresso em qualquer disciplina envolve construir modelos que possam ser testados por meio de estudos experimentais, para verificar se o entendimento atual sobre o campo de estudo é correto.

Vale ressaltar que, no intuito de realizar pesquisa científica em ES, devem ser consideradas metodologias usadas para pesquisa em diversas disciplinas, devido à inerente característica interdisciplinar da ES, que percorre desde aspectos técnicos até sociais e psicológicos ([Wohlin et al., 2000](#)). Dentre as metodologias, [Basili et al. \(1999\)](#) apresenta o paradigma experimental como o mais adequado para a adoção na tentativa de entender o processo de software, o produto e o contexto em que se desenvolve. E segundo [Wohlin et al. \(2000\)](#), os tipos de estudos experimentais em ES são: pesquisa de opinião (*survey*), estudo de caso, e experimento controlado.

Organização do capítulo

2.1 Experimento controlado

A estratégia de estudo cujos conceitos são descritos na *EXPEROntology* corresponde ao experimento controlado. Em uma visão geral, é construído e avaliado um modelo que representa profissionais aplicando métodos, técnicas e ferramentas durante atividades de desenvolvimento de software.

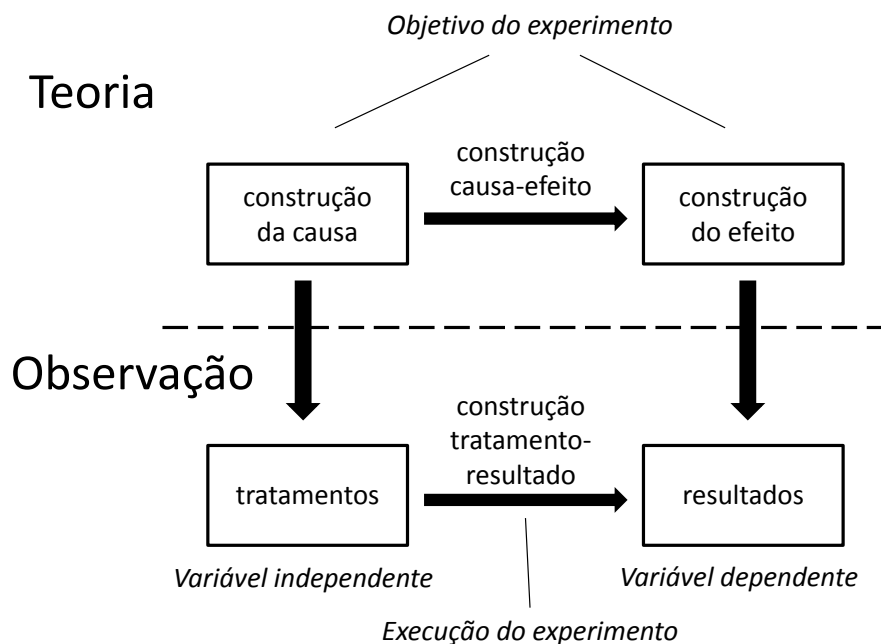


Figura 2.1: Relação causal avaliada em um experimento controlado (Wohlin et al., 2000)

Assim, os participantes selecionados os aplicam sob condições controladas, produzindo resultados cuja análise leva a conclusões sobre a tecnologia em estudo.

Para avaliar uma tecnologia em um experimento, é preciso isolar os fatores que influenciam sua aplicação e, de alguma maneira, captar os efeitos obtidos após aplicá-la. Assim, o ponto de partida é a ideia de uma relação de causa e efeito em teoria (Wohlin et al., 2000). Essa relação é formalizada em uma hipótese (ou um conjunto de hipóteses), cuja validade é testada durante o processo.

A relação causal (expressa na hipótese) é avaliada por meio da execução do experimento (vide Figura 2.1). Os elementos que influenciam a aplicação da tecnologia sob estudo (causa) são isolados e controlados para que por meio da análise dos resultados dessa aplicação (efeito) seja possível elaborar conclusões sobre o uso da tecnologia.

Os elementos que influenciam a aplicação da tecnologia são as **váriáveis independentes**. Um experimento estuda o efeito da mudança de uma ou mais variáveis independentes, os fatores (Wohlin et al., 2000). Os valores que um fator pode assumir são os **tratamentos**. Por exemplo, considerando um experimento cujo objetivo é analisar técnicas de Verificação e Validação (Basili e Selby, 1987), podem ser isolados os fatores *tipo de técnica* aplicada e *experiência dos participantes*. Para o fator *tipo de técnica*, os tratamentos podem ser: *leitura de código*, *teste estrutural* e *teste funcional*.

Conforme a atribuição de tratamentos, o efeito obtido pela execução varia. O efeito é representado por meio da **variável dependente**, cujos valores correspondem aos resultados da execução de um experimento. No caso do exemplo, a variável dependente pode ser número de defeitos encontrados (por um participante utilizando uma determinada técnica). Em suma, as variáveis independentes são as entradas da execução do experimento. E, de acordo com os valores que assumem, o efeito das mudanças refletem no valor da variável dependente (variável de saída), assim como está esquematizado na Figura 2.2.

Os tratamentos são aplicados a uma combinação de **objetos** e **participantes**. Um objeto pode

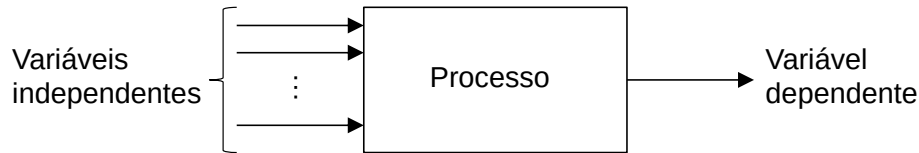


Figura 2.2: Variáveis independentes e dependentes (Wohlin et al., 2000)

ser, por exemplo, os programas em que devem ser aplicadas as técnicas de V&V. Os participantes são as pessoas que aplicam os tratamentos. A execução do experimento consiste na execução de um conjunto de **testes** (ou *trials*), em que cada teste é uma combinação de tratamento, participante e objeto (Wohlin et al., 2000). Por exemplo, um teste pode ser a pessoa N (*participante*) usar um método M (tratamento) para inspecionar o programa P (objeto). A partir da execução dos testes, são coletados os resultados do experimento.

Um aspecto fundamental que deve ser considerado é a ameaça à **validade** dos resultados. De acordo com Wohlin et al. (2000), há quatro tipos de validade: de conclusão, interna, de construção e externa. A *validade de conclusão*, cuja ameaça pode ser a escolha de um teste estatístico inadequado, é relacionada à habilidade de se chegar a uma conclusão correta da relação entre os conjuntos de tratamentos e os resultados do experimento. A *validade interna* garante que os resultados sejam influenciados apenas por fatores que sejam controlados e medidos. A *validade de construção* confirma a coerência entre a teoria e a observação, ou seja, a relação causa-efeito está bem representada pelas variáveis escolhidas. A *validade externa* se refere à capacidade de se generalizar os resultados para a população de interesse, o que implica que os participantes designados a conduzir o experimento devem ser selecionados de forma a constituírem uma amostra representativa de tal população.

2.2 Processo de Experimentação

Há uma sequência de passos para determinar o relacionamento causa-efeito que se deseja estudar e estabelecer e analisar um modelo que o reflita em termos de fatores, variáveis dependentes e objetos. Portanto, um processo que indique como um experimento é realizado é necessário (Wohlin et al., 2000). Adicionalmente, uma abordagem sistemática deve ser adotada com a finalidade de alcançar o nível de controle que um experimento requer. De acordo com a Figura 2.3, o processo sugerido por Wohlin et al. (2000) é composto pelas fases: **definição, planejamento, operação, análise e interpretação e apresentação e empacotamento**.

A *definição* estabelece o escopo do experimento, que é definido em termos dos objetivos e metas. No *planejamento*, as hipóteses são formalizadas e o projeto é determinado (o arranjo de quais participantes devem aplicar quais tratamentos). A *operação* é a execução do experimento propriamente dita. Nessa fase as medidas dos resultados são coletadas e então avaliadas na fase de *análise e interpretação*. Por fim, os procedimentos adotados, resultados e conclusões obtidos são armazenados na fase de *apresentação e empacotamento*.

2.2.1 Definição

A fase de definição fornece a direção geral do experimento, estabelecendo o seu escopo, suas metas e, conseqüentemente, a base para a formulação das hipóteses. As metas do experimento podem ser definidas de acordo com o seguinte modelo:

Analisar <Objeto(s) do estudo>
com o propósito de <Propósito>
com respeito a <Foco de qualidade>
do ponto de vista de <Perspectiva>
no contexto de <Contexto>

O *objeto do estudo* é a entidade que é estudada no experimento, que pode ser produtos, processos, recursos, modelos, métricas ou teorias. Um exemplo de objetos de estudo são técnicas de teste para serem aplicadas durante a verificação de qualidade. O *propósito* do experimento define o objetivo em termos gerais. Pode ser, por exemplo, comparar diferentes técnicas. O *Foco de qualidade* é o efeito sob estudo no experimento. Alguns exemplos são: efetividade, custo, confiabilidade, etc. A *perspectiva* revela o ponto de vista pelo qual os resultados do experimento são interpretados (desenvolvedor, gerente de projeto, cliente ou pesquisador). O *contexto* é o composto pela caracterização dos participantes (experiência, tamanho de equipe, etc) e dos objetos, ou seja, dos artefatos que devem ser usados no experimento (complexidade, tamanho, domínio de aplicação, etc).

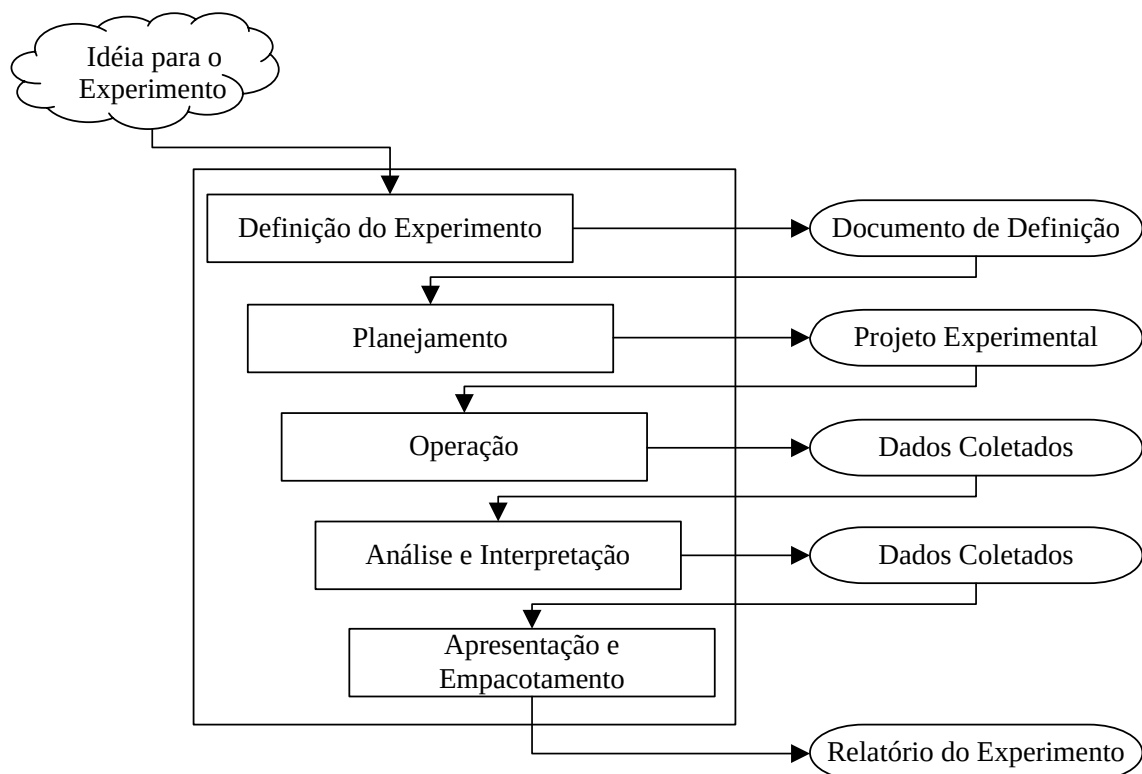


Figura 2.3: Processo de experimentação (Wohlin et al. (2000); Garcia (2006))

2.2.2 Planejamento

O planejamento estabelece como o experimento deve ser conduzido. A partir da definição das metas do estudo, essa fase tem como produto o projeto experimental. Para isso é necessário isolar as variáveis, formular as hipóteses, e arranjar os tratamentos, objetos e participantes de modo a organizar como os testes devem ser executados e os resultados analisados.

Em primeiro lugar, o **contexto** do experimento (que foi apontado na fase de definição) deve ser refinado. É preciso decidir se o experimento vai ser executado em um ambiente real ou simulado e se os participantes devem ser profissionais ou estudantes, por exemplo. Tais decisões envolvem uma análise dos riscos e custos envolvidos.

Em seguida, as **hipóteses** que expressam a relação de causa e efeito a ser estudada são formuladas. É o teste estatístico das hipóteses que apóia a análise dos resultados. Em geral, uma hipótese estatística é formulada unicamente com o propósito se ser rejeitada (Juristo e Moreno, 2001). Portanto, se a meta é comparar uma nova técnica de inspeção em relação a uma técnica antiga, a *hipótese nula* (H_0) afirma que não há diferença na média de defeitos encontrados por meio de ambas ($H_0 : \mu_{N_{antiga}} = \mu_{N_{nova}}$). Essa é a hipótese que o experimentador quer rejeitar com o maior nível de confiança possível. Uma *hipótese alternativa* (H_a) é a favor de que a hipótese nula seja rejeitada. No caso do exemplo, a hipótese alternativa afirmaria que a nova técnica encontra em média mais defeitos do que a técnica antiga ($H_a : \mu_{N_{antiga}} < \mu_{N_{nova}}$).

Uma vez que a relação sob estudo tenha sido expressa formalmente por meio das hipóteses, as **variáveis** que representam a causa e o efeito são selecionadas. As *variáveis independentes* são aquelas que influenciam no fenômeno (causa) e que se pode controlar durante o experimento. Entre as variáveis independentes, algumas são mantidas a níveis fixos e outras (os *fatores*) podem receber diferentes valores (os *tratamentos*), que também devem ser determinados. Quanto à *variável dependente*, deve ser derivada diretamente da hipótese e a medida escolhida para representá-la deve ser determinada. Por exemplo, a eficácia de uma técnica de inspeção pode ser medida pela média de defeitos encontrados pelos participantes.

Os **participantes** devem ser selecionados como uma amostra representativa da população à qual se deseja generalizar os resultados do experimento. Aspectos como o tamanho da amostra e a randomização devem ser considerados. Quanto maior e mais randômica a amostra de pessoas selecionadas, o erro ao generalizar os resultados se torna menor.

Após selecionar as variáveis e os participantes, é possível elaborar o projeto experimental. O projeto descreve como os testes devem ser organizados e executados durante a operação do experimento. A análise estatística a ser aplicada nos resultados depende da escolha do projeto. Para projetar o experimento, a partir da hipótese é preciso determinar qual análise estatística deve ser realizada para rejeitar a hipótese nula. No projeto também é estabelecida a quantidade de testes que são necessários para garantir que o efeito dos tratamentos seja visível. Nas tabelas 2.1 e 2.2 são apresentados exemplos de projetos com um fator e com dois fatores, respectivamente.

Antes da execução, é feita a **instrumentação** do experimento. Os *instrumentos* podem ser objetos, diretrizes e instrumentos de medida. Os objetos do experimento podem ser, por exemplo, documentos de especificação ou código. Tais objetos devem estar preparados para a execução do experimento. Por

Tabela 2.1: Exemplo de atribuição de participantes a tratamentos [Wohlin et al. \(2000\)](#)

Participantes	Tratamento 1	Tratamento 2
1	X	
2		X
3		X
4	X	
5		X
6	X	

Tabela 2.2: Exemplo de um projeto com dois fatores [Wohlin et al. \(2000\)](#)

		Fator A	
		Tratamento A1	Tratamento A2
Fator B	Tratamento B1	Participantes: 4,6	Participantes: 1,7
	Tratamento B2	Participantes: 2,3	Participantes: 5,8

exemplo, se o experimento investiga técnicas de inspeção, os defeitos devem ser inseridos nos objetos a serem inspecionados. *Diretrizes* são necessárias para guiar os participantes no experimento (além de um treinamento no método a ser usado). As medições em um experimento são conduzidas por meio da coleta de dados, o que geralmente é feito por meio de formulários ou entrevistas.

Por fim, também é importante considerar a validade dos resultados. É preciso analisar as ameaças desde antes da execução, planejando para obter uma validade adequada dos resultados. Há quatro tipos de validade que se deve assegurar em um experimento: de conclusão, interna, de construção e validade externa. Uma lista de ameaças à validade de um experimento é dada em [Wohlin et al. \(2000\)](#) e pode ser usada como uma *checklist* durante a fase de planejamento.

2.2.3 Operação

Uma vez que o experimento foi planejado, deve ser executado para que seus resultados sejam analisados. A fase de operação envolve a execução de fato do experimento. Como os estudos em ES lidam com pessoas, elas devem ser devidamente motivadas para se empenharem nas atividades, para garantir que os resultados obtidos sejam válidos.

Durante a operação os participantes devem ser selecionados e informados a respeito do experimento. É importante que as pessoas estejam motivadas e dispostas a participar ao longo do experimento. Todos os instrumentos devem estar prontos: as diretrizes devem ser passadas e os formulários entregues aos participantes.

A execução do experimento pode ser realizada em reuniões em que os participantes aplicam os tratamentos e geram dados dados. Esses resultados são coletados manualmente em formulários, com o apoio de ferramentas ou por meio de entrevistas. Quando os dados são coletados, o pesquisador precisa verificar se os dados são razoáveis e se foram coletados corretamente.

2.2.4 Análise e interpretação

Nessa fase os resultados da execução do experimento são analisados para que seja possível obter conclusões sobre a tecnologia em estudo. Os dados podem ser interpretados em três passos: estatística

descritiva, redução do conjunto de dados e teste da hipótese.

Após coletar os dados, a estatística descritiva pode ser usada para apresentar graficamente aspectos dos dados, como a tendência central e a dispersão. Alguns gráficos que podem ser utilizados são: gráfico de dispersão (*scatter plot*), histograma e gráfico de setores. Os gráficos provêm um meio de analisar os dados e identificar distorções (*outliers*). A partir dos gráficos gerados, é possível detectar distorções (*outliers*), ou seja, pontos de dados que são muito maiores ou muito menores do que os outros. Quando *outliers* são identificados, é preciso analisar as razões que os ocasionaram para decidir se devem ser eliminados ou não.

Em seguida, a hipótese estabelecida na fase de planejamento deve ser testada com base nos dados gerados pela execução do experimento. O teste da hipótese rejeitar a hipótese nula, H_0 , com base em uma amostra de alguma distribuição estatística. A escolha do teste estatístico que pode ser aplicado depende de como o experimento foi projetado. Algumas opções de testes estatísticos podem ser vistas na Tabela 2.3.

Tabela 2.3: Visão geral de testes paramétricos/não-paramétricos para diferentes projetos [Wohlin et al. \(2000\)](#)

Projeto	Paramétricos	Não-paramétricos
Um fator, um tratamento		Chi-2 Teste binomial
Um fator, dois tratamentos, projeto completamente aleatório	teste t F-test	Mann-Whitney Chi-2
Um fator, dois tratamentos, comparação pareada	teste t pareado	Wilcoxon Teste de sinal
Um fator, mais de dois tratamentos	ANOVA	Kruskal-Wallis Chi-2
Mais de um fator	ANOVA	

Por fim, se a hipótese nula foi descartada é possível obter conclusões sobre a influência das variáveis independentes nas variáveis dependentes, considerando que o experimento é *válido*. Após descartar a hipótese, é preciso tirar conclusões gerais sobre a relação entre as variáveis, considerando a *validade externa* do experimento. Se o experimento não pôde rejeitar a hipótese nula não é possível afirmar algo sobre a relação entre as variáveis.

2.2.5 Empacotamento

A fase de empacotamento tem como objetivo documentar todo o processo, gerando como produto um pacote de laboratório. Uma documentação completa e detalhada é necessária para, além de apresentar os resultados do estudo, facilitar a sua replicação. Por isso, conforme o esquema da Figura 2.4, a melhor abordagem para a fase de empacotamento é que seja executada ao longo do processo de experimentação e não apenas como etapa final ([Amaral e Travassos, 2003](#)), o que aumenta a fidelidade do pacote de laboratório à condução do experimento ([Garcia, 2006](#)). Portanto, a cada fase devem ser geradas as informações correspondentes.

2.3 Diretrizes para reportar experimentos controlados

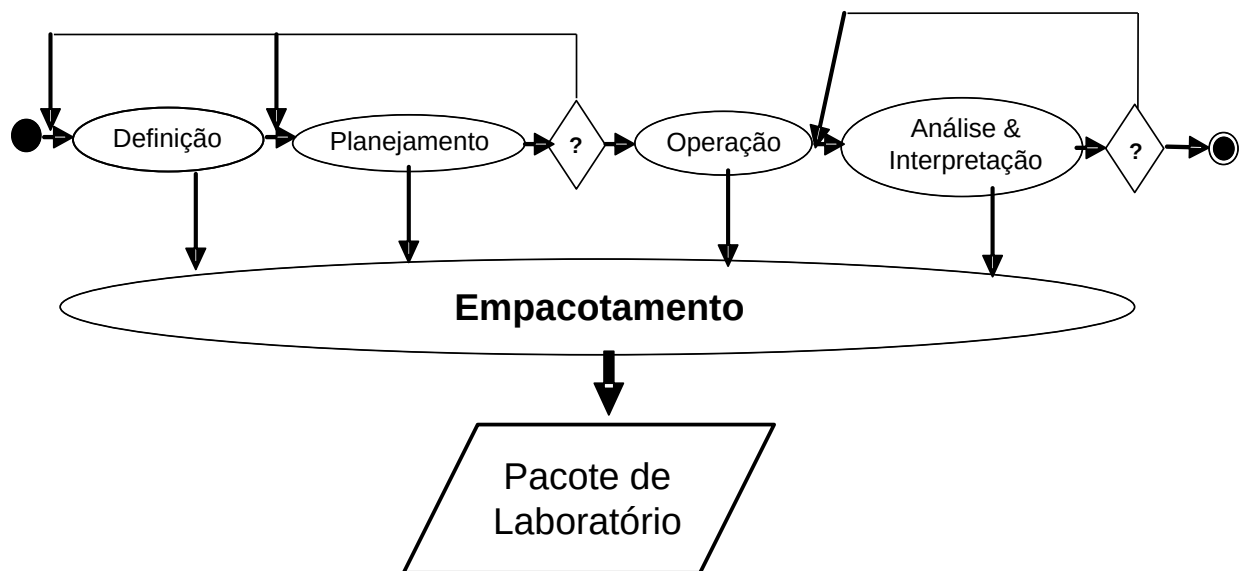


Figura 2.4: Processo de experimentação - fase de empacotamento (Wohlin et al. (2000); Amaral e Travassos (2003))

Tabela 2.4: Visão geral das propostas de diretrizes de empacotamento (Jedlitschka et al., 2008)

	Singer (1999)	Wohlin et al. (2000)	Kitchenham et al. (2002)	Juristo e Moreno (2001)	Kitchenham (2004)	Jedlitschka et al. (2007)
tipo de estudo	pesquisa experimental	pesquisa experimental	pesquisa experimental	experimento controlado	revisão sistemática	experimento controlado
fases do estudo	empacotamento	todas	todas	todas	todas	empacotamento
estrutura	*	*	*	*	*	título
	*	*	*	*	autores	autores
	*	*	*	*	palavras-chave	palavras-chave
	resumo	*	*	*	resumo estruturado	resumo estruturado
	introdução	introdução	*	definição das metas	<i>background</i>	introdução
		declaração do problema				
		planejamento do experimento	contexto experimental		<i>background</i>	<i>background</i>
	introdução	declaração do problema	contexto experimental	definição de metas	revisão de questões	planejamento do experimento
	método	planejamento do experimento	projeto experimental	projeto	revisão de métodos	desvios do planejado
	procedimento	operação do experimento	condução do experimento e coleta de dados	execução do experimento	estudos incluídos e excluídos	
	resultados	análise de dados	análise	análise experimental	resultados	análise
	discussão	discussão e conclusões	*	análise experimental	conclusão	conclusões e trabalhos futuros
	-	-	-	-	agradecimentos	agradecimentos
	referências	referências	*	*	conflito de interesses	referências
	apêndices	apêndice	*	*	apêndices	apêndices

Os asteriscos (*) indicam que os elementos são implicitamente requeridos, apesar de não serem explicitamente mencionados.

2.4 Replicações e Transferência de Conhecimento

A replicação de experimentos em diferentes contextos torna possível verificar a validade de uma hipótese com um nível de confiança maior, pois o experimento está sujeito a diversas variações, como o ambiente cultural em que é executado ou a habilidade e experiência dos participantes selecionados (Basili et al. (1999); Shull et al. (2003); Mendonça et al. (2008)). E, considerando tais variações, restringe-se a generalização das conclusões do estudo, uma vez que as condições impostas durante sua execução limitam a população à qual se aplicam as conclusões. Assim, os resultados experimentais devem ser reproduzíveis externamente ou a influência dessas variações deve ser estudada. Por meio de outros pesquisadores reproduzindo com sucesso um experimento, a confiança nos resultados se constrói (Brooks et al., 2008).

Para possibilitar a condução de replicações e o uso do conhecimento adquirido em cada experimento é fundamental registrar de forma precisa todo o procedimento de condução do experimento no pacote de laboratório (Garcia, 2006). O pacote de laboratório deve ser um artifício para lidar com a evolução do experimento através de múltiplos estudos.

Entretanto, Shull et al. (2002) apontaram que, apesar de disponíveis, pesquisadores enfrentam dificuldades em entender e revisar pacotes de laboratório. Os replicadores podem pertencer a um contexto diferente daquele dos projetistas do experimento original, podem ter uma perspectiva diferente. Por um lado, o fato de pesquisadores de contextos diferentes replicarem um estudo contribui para a capacidade de generalização de resultados obtidos, o que é fundamental, pois é justamente a validade externa dos resultados que permite sua aplicação efetiva por engenheiros de software. Contudo, o contexto diferente entre pesquisadores do experimento e os de sua replicação pode dificultar o entendimento do pacote de laboratório.

Mendonça et al. (2008), baseados na experiência de replicar experimentos, chamaram a atenção para a dificuldade de se transferir o conhecimento adquirido em experimentos e propuseram o **FIRE** (*Framework for Improving the Replication of Experiments*), um arcabouço composto de atividades que visam a contornar aspectos que possam dificultar a replicação.

O FIRE é composto por dois ciclos, conforme ilustrado na Figura 2.5. O ciclo interno engloba atividades relacionadas à execução de um experimento isolado (planejamento, execução, empacotamento) em um grupo de pesquisa. O ciclo externo lida com a aprendizagem entre os grupos de pesquisa e visa a construir um amplo corpo de conhecimento a partir de replicações cuidadosamente planejadas. Assim, trata de questões como a padronização e o entendimento de pacotes de laboratório, que contribuem na transferência de conhecimento a grupos de outros contextos, dispostos a replicar o estudo.

Compartilhar conhecimento se configura nas atividades de interface entre os ciclos do FIRE e é um ponto chave para se empreender replicações. Mendonça et al. (2008) salientam que o foco não pode ser dado apenas ao ciclo interno, ou então os estudos correm o risco de se tornarem isolados. É preciso integrá-los em um corpo de conhecimento.

Um dos fatores mais importantes para contribuir com o compartilhamento de pacotes é a padronização do conjunto de informações que devem conter, e, além disso, a sua evolução na direção deste padrão (também atividade do FIRE). Jedlitschka et al. (2008) confirma a relevância da padroniza-

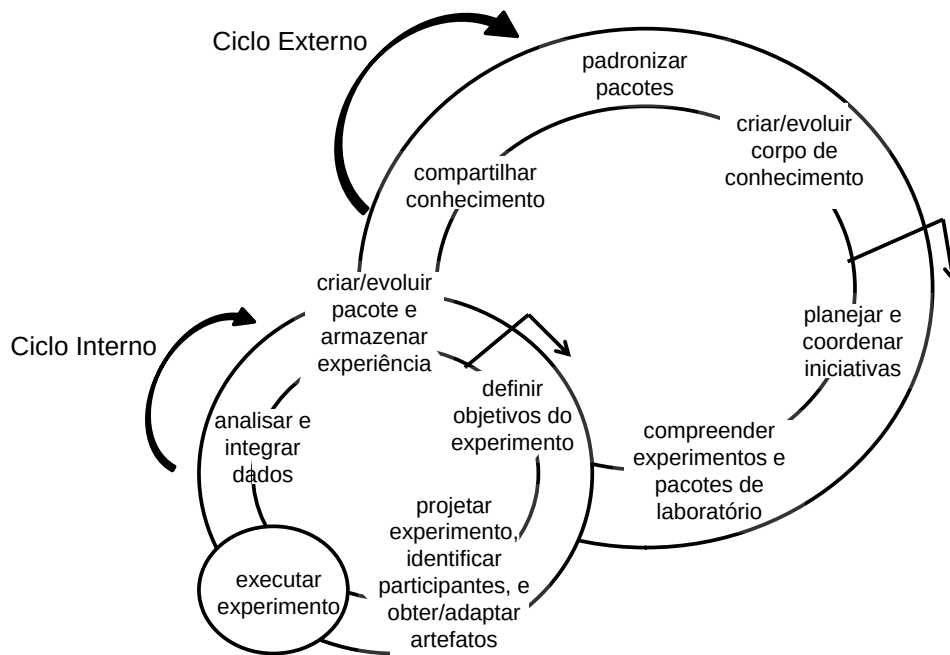


Figura 2.5: Processo de experimentação (Mendonça et al., 2008)

ção ao evidenciar que a irregularidade das maneiras de se reportar estudos é um grande problema encontrado para se integrar os respectivos resultados em um corpo comum.

Carver (2010), ao fazer uma revisão entre publicações de experimentos, propôs diretrizes tendo a padronização como meta, após constatar que as informações de estudos não eram publicadas de maneira padrão, tanto no tipo de informação contida, quanto no nível de detalhes. Como exemplos de publicações contendo informações diferentes pode-se citar Xie et al. (2008), Collins et al. (2008) e Lucia et al. (2010), além dos estudos revisados por Carver (2010).

2.5 A ExperOntology

A *EXPEROntology* (Garcia et al., 2008) foi elaborada para facilitar o reuso e o compartilhamento de pacotes de laboratório. Basicamente, a ontologia esclarece os conceitos associados a experimentos controlados e suas relações. São definidos conceitos de todo o processo de experimentação, desde a definição até a análise dos resultados. Assim, o empacotamento de um estudo é dado pelos conceitos da ontologia instanciados.

A captura da conceituação da ExperOntology, contida parcialmente nas figuras 4 e 5, é apresentada por Garcia et al. (2008) em dois níveis de refinamento. O primeiro nível (Figura 4) se refere aos principais conceitos de um experimento controlado. O segundo (Figura 5) se refere ao refinamento do conceito de pacote de laboratório. A captura dos conceitos foi representada usando a linguagem UML.

Para formalizar a ontologia, foram usados axiomas de lógica de primeira ordem. O predicado *Design*, que associa tratamentos a cada participante do experimento, é dado por:

$$Design(s, SetOfTreat)$$

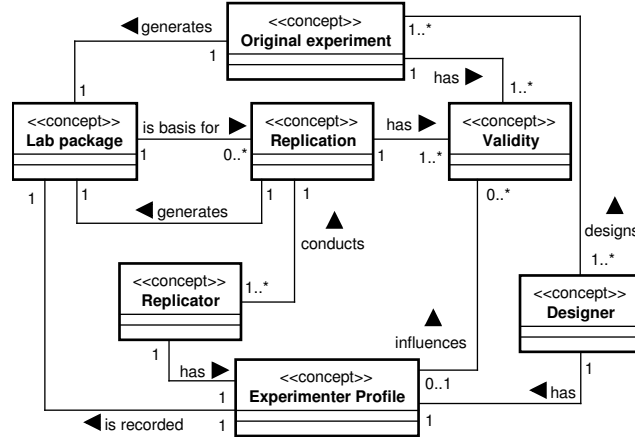


Figura 2.6: Conceitos de um experimento controlado (Garcia et al., 2008)

em que s representa os participantes e $SetOfTreat$ são os valores que os fatores podem assumir (conjunto de tratamentos). Fatores, seus possíveis tratamentos e um conjunto de tratamentos são definidos por:

$$\begin{aligned}
 &Factor(f_1, \dots, f_n) \\
 &\forall f, \exists Treatment(f) = v_1, \dots, v_n, n \geq 2 \\
 &dom(Treatment) = Artifact \cup Technology \\
 &SetOfTreat = (vf_1, \dots, vf_n) \mid \forall f, vf_n \in Treatment(f_n)
 \end{aligned}$$

em que f define um fator do experimento, e vf_n são tratamentos atribuídos para cada f . A partir do predicado *Design* é estabelecido o plano para a condução do experimento, o qual é detalhado em um conjunto de tarefas:

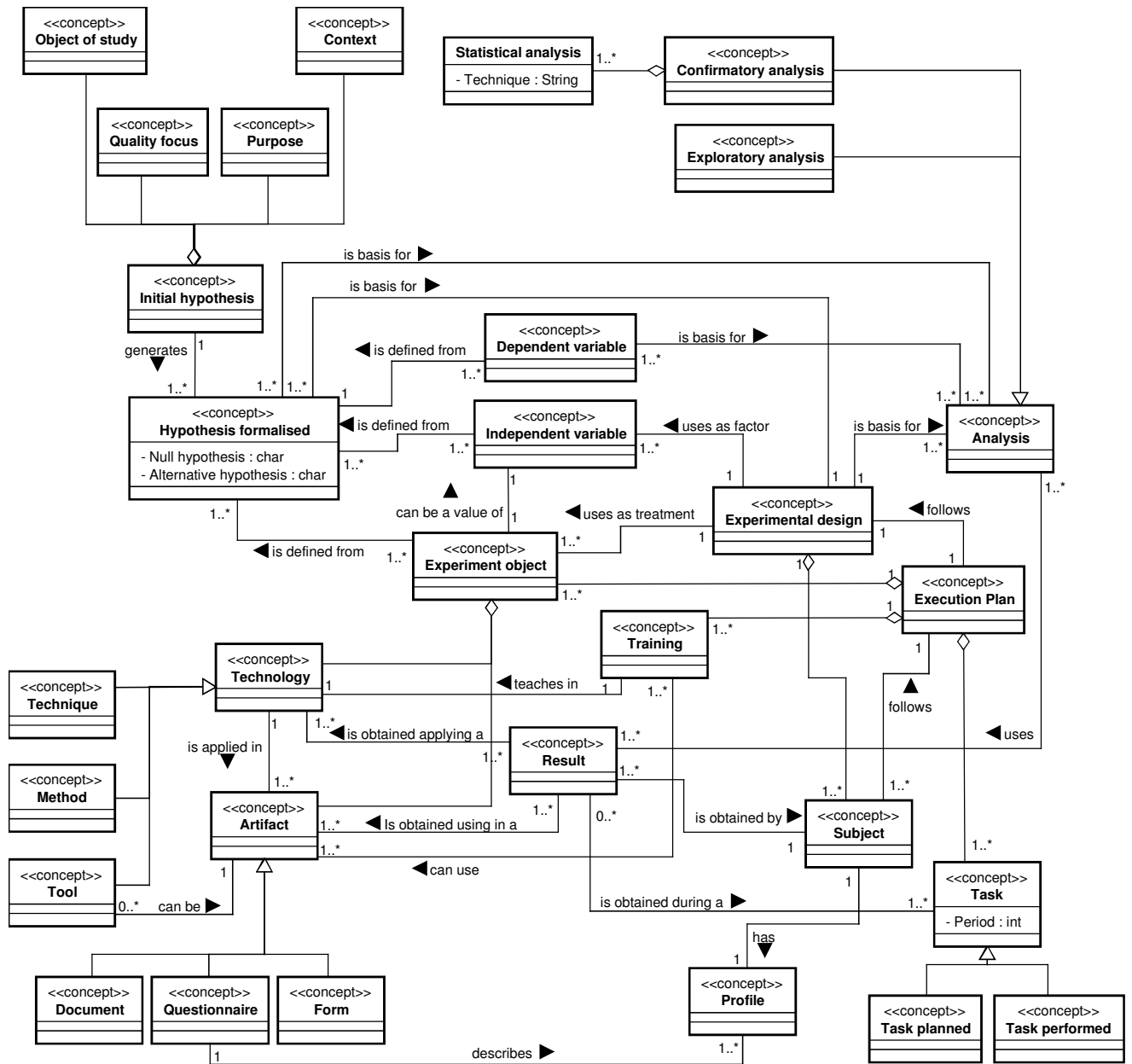
$$(\forall s, SetOfTreat)(Execution(s, SetOfTreat))$$

em que ta_n define as tarefas que devem ser seguidas durante a fase de operação. Por sua vez, cada tarefa é definida como:

$$Task(ta_n) \rightarrow (Training(s, t_r, a, p) \wedge Applying(s, t_e, a, p))$$

em que o predicado *Training* indica que um participante s pode ser treinado em uma tecnologia t_r usando o artefato a durante um período de tempo p . De forma semelhante, *Applying* indica que um participante s aplica a tecnologia t_r no artefato a durante um período de tempo p . Vale lembrar que uma tarefa de aplicação gera resultados para amparar o teste da hipótese.

O objetivo em usar a *EXPEROntology* ao empacotar experimentos é incorporar à transferência de conhecimento por meio de pacotes de laboratório as vantagens que ontologias fornecem às suas aplicações: favorecer o entendimento compartilhado das informações (Noy e McGuinness, 2001), para que, assim, a transferência de conhecimento seja efetiva.



Matching de esquemas/ontologias

Não há um padrão amplamente adotado para se reportar experimentos em ES, apesar de terem sido propostas diretrizes com o objetivo de lidar com a falta de padronização na pesquisa da área (Jedlitschka et al., 2008, Kitchenham et al., 2002). Os estudos experimentais já realizados formam um corpo de conhecimento heterogêneo, em que se torna difícil localizar informações relevantes e, em muitos casos, algumas informações podem estar ausentes no relatório de um experimento (Jedlitschka et al., 2008).

Por isso há o interesse em empacotar os estudos de acordo com um padrão, para viabilizar a integração dos resultados e facilitar a revisão dos pacotes de laboratório. Dessa maneira, é favorecida a realização de replicações, uma vez que é por meio do pacote de laboratório que se compreende o experimento original a fim de estabelecer as metas da replicação.

Os pacotes de laboratório já existentes apresentam conjuntos de informação diferentes e, portanto, apresentam esquemas de dados diferentes. Esquemas são definições sobre a estrutura dos dados, como por exemplo, esquemas de banco de dados, esquemas XML ou até mesmo ontologias formais. Cada esquema é composto por um conjunto de elementos de dados e, possivelmente, as relações que existem entre esses elementos.

Considerando que a *EXPEROntology* deve evoluir em um padrão semântico que acomode diferentes estruturas de pacote de laboratório, é preciso determinar, para cada estrutura distinta, quais são os elementos em comum com os conceitos da ontologia, a fim de instanciá-los, e quais são os elementos da estrutura que não possuem conceito correspondente, para que os conceitos faltantes sejam incorporados na ontologia. O processo de encontrar essas correspondências é chamado de *matching* de esquemas (Rahm e Bernstein (2001)).

Neste capítulo é dada uma visão geral sobre *matching*. Na Seção 3.1 são apresentados conceitos básicos e o *matching* é definido em termos de suas entradas (dois esquemas/ontologias) e saídas (correspondências entre seus elementos). Também são comparadas as características de *matching* de esquemas e de ontologias, no sentido de esclarecer se há diferenças nas técnicas que devem ser aplicadas nos dois casos. Em seguida, na Seção 3.2, o *matching* é apresentado como um processo

composto por uma sequência de fases e funcionamento de cada uma delas é detalhado.

3.1 Definições gerais e operação de *matching*

Quando esquemas distintos são elaborados para um mesmo domínio, quase sempre são diferentes uns dos outros (Halevy, 2005). Dois elementos, cada um pertencente a um esquema distinto, podem ter o mesmo significado, mas nomes diferentes. Alternativamente, outros dois elementos podem ter nomes idênticos e apresentarem significados incompatíveis. Portanto, se for preciso haver interoperabilidade entre aplicações que usam dois esquemas distintos, a detecção de discrepâncias e de similaridades entre os elementos dos esquemas deve ser considerada (Hakimpour e Geppert, 2001).

Para que os sistemas cooperem uns com os outros, devem entender os esquemas uns dos outros. Projetistas de sistemas distintos geralmente têm necessidades diferentes e, portanto, elaboram esquemas heterogêneos. Dessa maneira, o problema deve ser resolvido ao passo que o fornecedor dos dados modelados pelo esquema os expõe aos outros sistemas interessados (Halevy, 2005).

Esse é o mesmo caso do problema de heterogeneidade nos repositórios de conhecimento de ES Experimental, em que os fornecedores de dados (os grupos de pesquisa de ES) empacotam estudos de acordo com esquemas distintos, sendo que esses pacotes de laboratório precisam ser integrados em um corpo comum para evitar que os estudos se tornem isolados. É nesse sentido que a *EXPEROntology* pode utilizada como padrão para instanciar os estudos com diferentes conjuntos de informação. Pode ser feito o *matching* dos esquemas dos pacotes provenientes de grupos de pesquisa distintos com a ontologia, com o intuito de instanciá-los em uma mesma estrutura de informação.

A abordagem comum para esse problema é a definição de relações entre os elementos das representações heterogêneas (Bouquet et al., 2004). Alguns desafios para resolvê-lo estão ilustrados na Figura 3.1, em que se pode observar o instantâneo da interface de uma ferramenta de *matching* manual de esquemas. O projetista deve desenhar linhas (representando as relações) entre os elementos correspondentes dos dois esquemas (Halevy, 2005). Nesse exemplo são mostrados alguns tipos de discrepâncias que podem existir: (i) elementos podem apresentar nomes diferentes (*IssueDate* e *OrderIssueDate*), ou (ii) podem ser agrupados (ou aninhados) de diferentes maneiras (subárvores de *BuyerParty*), e (iii) um esquema pode abranger aspectos do domínio não abrangidos pelo outro (o esquema da esquerda não apresenta o elemento *OrderSummary* como o da direita).

O *matching* de esquemas é esse processo de encontrar relações entre os fragmentos de informação, que são modelados usando diferentes estruturas e formatos. A partir do exemplo da Figura 3.1, mesmo apresentando esquemas de pequena dimensão, já se pode constatar que o *matching* de esquemas de maneira puramente manual é tediosa e demorada. E, conforme a dimensão dos esquemas aumenta, o esforço requerido torna-se proibitivo. Resolver essas tarefas em larga escala requer abordagens ao menos semi-automáticas de *matching* de esquemas (Rahm, 2011). Basicamente, são tomados como entrada dois esquemas, cada um consistindo de um conjunto de entidades (por exemplo: tabelas, elementos XML, classes, propriedades, regras, predicados) e determina-se como saída as relações (por exemplo: equivalência) que existem entre essas entidades (Shvaiko e Euzenat, 2005).

A definição de uma correspondência estabelecida entre dois elementos de duas ontologias o e o' pode ser formalizada pela estrutura (Shvaiko e Euzenat, 2012):

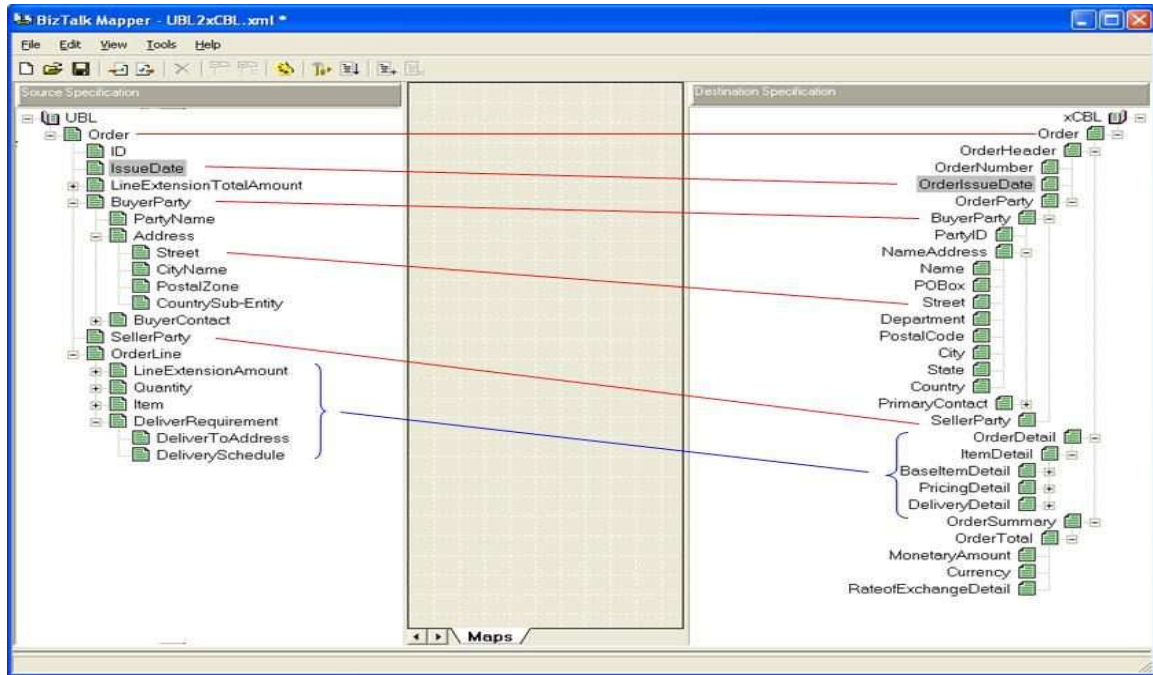


Figura 3.1: Conciliando dois esquemas diferentes (Halevy, 2005)

$$\langle e, e', r, n \rangle$$

em que:

- e e e' são entidades, como por exemplo, classes e propriedades, de o e o' respectivamente;
- r é uma relação, como por exemplo, equivalência ($\equiv$), subsunção ($\sqsubseteq$), disjunção ($\perp$), que existe entre e e e' ;
- n é uma medida de confiança (tipicamente do intervalo $[0, 1]$) para a correspondência entre e e e' .

A correspondência $\langle e, e', r, n \rangle$ declara que a relação r existe entre as entidades e e e' com um nível de confiança n . Um exemplo simples de correspondência é (Euzenat e Shvaiko, 2007):

<http://book.ontologymatching.org/example/culture-shop.owl#Book> $=_{1.0}$
<http://book.ontologymatching.org/example/library.owl#Volume>

que declara a relação de equivalência (r) com confiança total ($n = 1$) entre duas entidades **Book** (e) e **Volume** (e'), pertencentes às ontologias *culture-shop.owl* (o) e *library.owl* (o').

Ao realizar a operação do *matching*, um conjunto de correspondências deve ser encontrado, provavelmente considerando o produto cartesiano dos conjuntos de entidades a serem comparadas das ontologias o e o' . Esse conjunto de correspondências é chamado de alinhamento entre as duas ontologias e consiste na saída da operação de *matching*. Considerando o exemplo das ontologias de biblioteca e de loja de livros, um alinhamento possível seria:

$\text{Book} =_{1.0} \text{Volume}$
 $\text{name} \geq_{1.0} \text{title}$
 $\text{id} \geq_{0.9} \text{isbn}$
 $\text{author} =_{1.0} \text{author}$

Uma visão geral da operação de *matching* pode ser observada na Figura 3.2: é determinado um alinhamento A' para um par de ontologias o e o' (Euzenat e Shvaiko, 2007). Há algumas outras entradas opcionais que podem estender essa definição, como (Shvaiko e Euzenat, 2012): (i) o uso de um alinhamento de entrada A (a ser estendido pelo processo), (ii) parâmetros para as técnicas de *matching* (pesos, *thresholds*) e (iii) recursos externos (por exemplo, um vocabulário específico do domínio de ambas as ontologias).

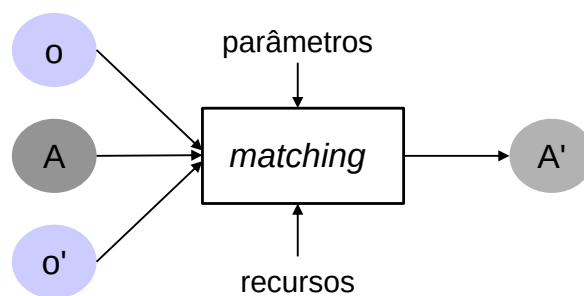


Figura 3.2: Operação de *matching* (Euzenat e Shvaiko, 2007)

Há diversos tipos de estruturas que podem ser as entradas da operação de *matching*, além de ontologias. Devem ser considerados também os outros esquemas ou modelos conceituais: esquemas de banco de dados, esquemas XML, diagramas UML, etc. Cada aplicação escolhe um desses tipos de esquema como a maneira de armazenamento de dados mais adequada. Portanto, quando se coordena ou integra dados de diferentes fontes, deve-se realizar o *match* entre vários tipos de modelos conceituais.

Por causa disso, na literatura são utilizados dois termos distintos: *matching* de esquemas (Rahm e Bernstein (2001); Shvaiko e Euzenat (2005); Rahm (2011)) e *matching* de ontologias¹ (Euzenat e Shvaiko (2007); Noy (2009); Shvaiko e Euzenat (2012)), cada um enfatizando o tipo de estrutura de entrada do *matching*. Entretanto, ambas as abordagens resolvem o mesmo problema: a heterogeneidade semântica (Hakimpour e Geppert (2001); Halevy (2005)).

Matching de esquemas e de ontologias compartilham algumas características comuns e diferenças (Shvaiko e Euzenat, 2005):

- Esquemas não provêm semântica explícita aos dados. Ontologias são especificações que obedecem a alguma semântica formal, ou seja, é possível interpretar as definições da ontologia como um conjunto de axiomas lógicos.
- Entretanto, ontologias e esquemas são similares no sentido que (i) ambos provêm um vocabulário de termos que descreve um domínio de interesse e (ii) ambos restringem o significado dos termos usados no vocabulário, mesmo que variando no grau de formalidade dessa restrição.

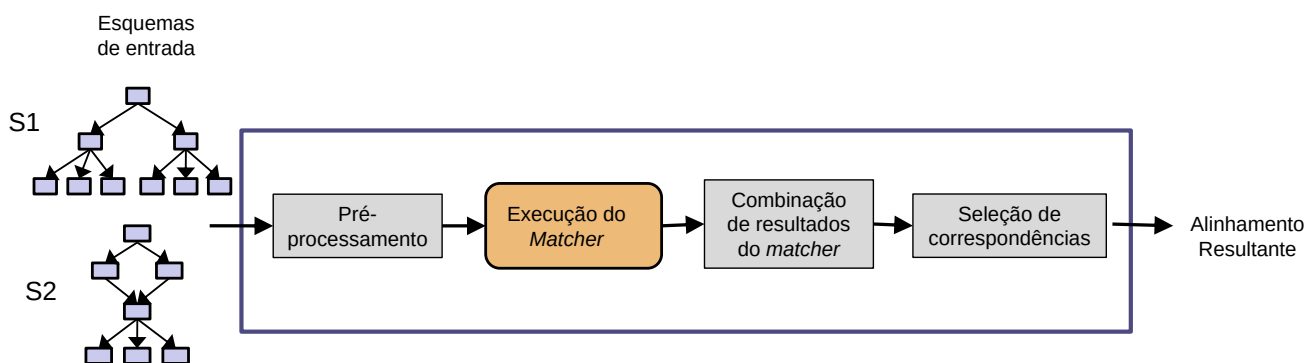
¹Também são utilizados os termos *mapeamento* e *alinhamento* de ontologias

As técnicas aplicadas a cada tipo de esquema deve tirar proveito dos elementos e estrutura de cada um. Uma ontologia também pode ser considerada um esquema, com a característica adicional de apresentar a descrição semântica das informações. *Matching* de ontologias abrange as técnicas de *matching* de esquemas e técnicas que consideram um modelo semântico.

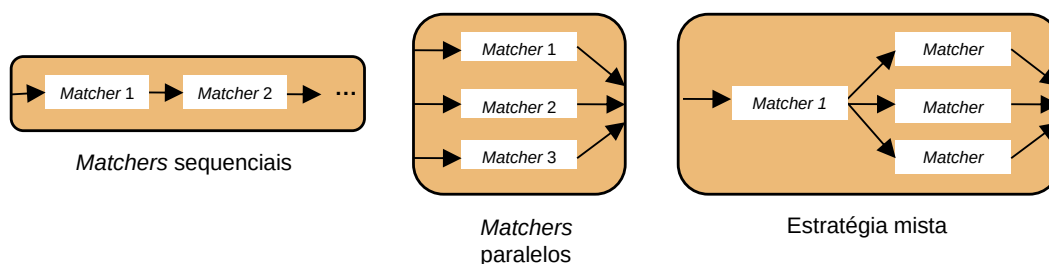
Assim, por um lado, o *matching* de esquemas geralmente é realizado com a ajuda de técnicas que tentam supor o seu significado. Por outro lado, sistemas de *matching* de ontologias podem, além de usar técnicas baseadas em esquemas, explorar o conhecimento explicitamente codificado nas ontologias. Mas, apesar de haver diferenças entre os dois casos, as técnicas desenvolvidas para cada um são de benefício mútuo (Shvaiko e Euzenat, 2005).

3.2 Processo de *matching*

Na Figura 3.2, o *matching* foi apresentado como uma caixa preta, uma operação que recebe entradas e devolve resultados. Focando nas partes internas, pode-se considerar o *matching* como um processo, composto de várias partes internas, conforme pode ser visto na Figura 3.3(a).



(a) Processo geral de *matching* de esquemas



(b) Composição de *matchers*

Figura 3.3: Processo de *matching* de esquemas (Rahm, 2011)

Esse é o processo geral para o *matching* automático de esquemas que vem sendo usado por muitos sistemas atuais (Rahm, 2011). Como primeiro passo, na etapa de *pré-processamento*, os elementos dos esquemas são postos em condições de serem analisados. Além disso, podem ser feitos outros tipos de processamento no sentido de preparar as estruturas de entrada para a execução propriamente dita.

Em seguida, diversos *matchers* individuais são executados e estabelecem um conjunto preliminar

de correspondências. Após a fase de execução, os resultados dos *matchers* são combinados e, por fim, há a seleção de correspondências do resultado combinado (Rahm, 2011).

3.2.1 Execução

A fase de *execução do matcher*, como ilustrado na Figura 3.3(b), consiste na execução de diversos *matchers* individuais. Cada *matcher* individual implementa uma técnica de *matching*. As técnicas têm o objetivo de encontrar as relações entre os elementos. Em geral, essas relações são descobertas por meio da medida de similaridade entre os elementos dos esquemas/ontologias (Euzenat e Shvaiko, 2007).

A técnica a ser aplicada em uma solução de *matching* depende do tipo de elementos dos esquemas de entrada, e mais especificamente, das características que esses elementos oferecem para serem comparadas entre si. Para realizar o *match* de duas classes de uma ontologia, por exemplo, algumas características que podem ser comparadas são os identificadores (nomes) das classes, os atributos (propriedades) de cada uma ou as relações mantidas com as outras classes. De acordo com o tipo de entrada do processo de matching o cálculo da similaridade deve ser implementado para se adequar às características fornecidas pelos elementos dos esquemas de entrada.

Essas técnicas podem ser combinadas (vide Figura 3.3(b)) a fim de a similaridade seja calculada com base em um conjunto abrangente de características dos elementos. Os *matchers* individuais, que implementam as técnicas, são as unidades de uma solução de *matching* completa e, portanto, é preciso adotar uma estratégia para organizar a sua execução.

Uma das estratégias de composição de *matchers* individuais é a *sequencial* (vide Figura 3.4), em que um *matcher* pode aproveitar e refinar os resultados do que foi executado anteriormente. Outra maneira é a composição paralela, em que as técnicas são executadas independentemente e seus resultados precisam ser agregados (vide Figura 3.5).

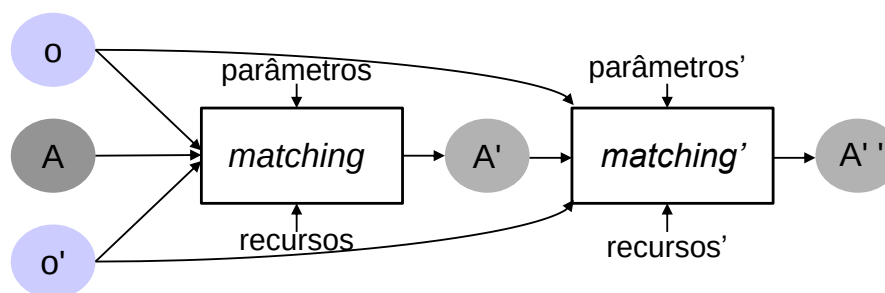


Figura 3.4: Composição sequencial de *matchers* (Euzenat e Shvaiko, 2007)

Considerando essas duas estratégias distintas, ainda é possível uma abordagem híbrida que reúne ambas as estratégias. No caso de haver a necessidade de agregar os resultados de técnicas executadas paralelamente, há a próxima etapa do processo que consiste na *combinação de resultados do matcher*.

3.2.2 Combinação de resultados

No caso da composição sequencial não há a necessidade da execução dessa fase pois o alinhamento retornado como saída por uma técnica é tomado como entrada pela técnica subsequente. Com essa abordagem, o alinhamento é aprimorado ao longo da sequência de técnicas de *matching*.

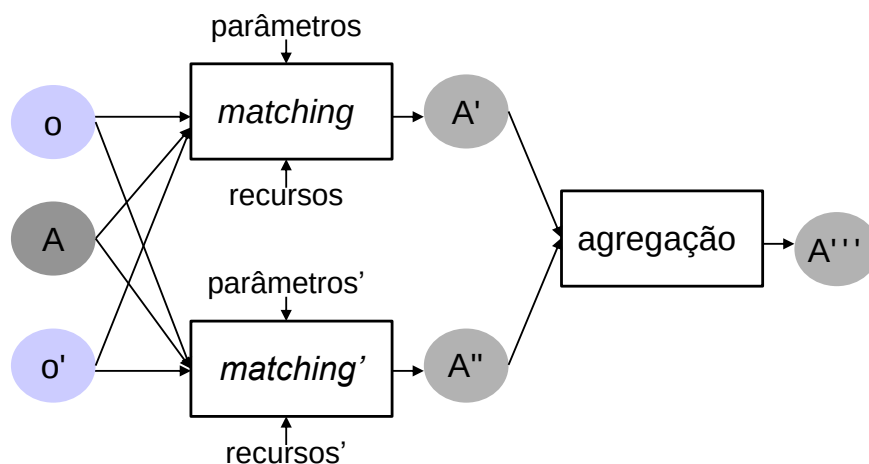


Figura 3.5: Composição paralela de *matchers* (Euzenat e Shvaiko, 2007)

Por outro lado, se os *matchers* são executados em paralelo é preciso agregar os resultados retornados por cada um de modo a determinar a similaridade entre os elementos sob comparação. Por exemplo, computar a similaridade entre duas classes requer a agregação (em uma única medida de similaridade) da similaridade obtida dos nomes, da similaridade de suas superclasses, da similaridade de suas propriedades e das suas instâncias (Euzenat e Shvaiko, 2007).

Na Figura 3.6 há duas matrizes de dissimilaridade que foram obtidas ao aplicar a distância de edição e a distância baseada no WordNet aos nomes dos conceitos de duas ontologias. Para agregar os resultados em apenas uma matriz, um dos métodos que podem ser aplicados é manter a distância mínima entre dois elementos correspondentes da matriz, o que resulta na matriz da Figura 3.7.

	Book	Translator	Publisher	Writer
Product	.86	.8	.89	.86
Provider	.88	.8	.56	.5
Creator	.86	.5	.89	.57

Normalised Levenshtein distance.

	Book	Translator	Publisher	Writer
Product	.82	.88	.88	.85
Provider	.83	.89	.76	.71
Creator	.82	.53	.88	.85

Alignment API WordNet-based distance.

Figura 3.6: Matrizes de dissimilaridade (Euzenat e Shvaiko, 2007)

	Book	Translator	Publisher	Writer
Product	.82	.8	.88	.85
Provider	.83	.8	.56	.5
Creator	.82	.5	.88	.57

Minimum of the distances.

Figura 3.7: Matriz resultante da agregação (Euzenat e Shvaiko, 2007)

Outra maneira de agregar os resultados das técnicas é a soma ponderada das similaridades obtidas

por cada técnica. Assim é possível aumentar a influência de técnicas mais precisas no resultado final. Se o é um conjunto de elementos que podem ser analisados por n técnicas distintas, a soma ponderada entre esses elementos é dada por:

$$\forall x, x' \in o, \delta(x, x') = \sum_{i=1}^n w_i \times \delta(x_i, x'_i)$$

em que $\delta(x_i, x'_i)$ é a dissimilaridade do par de objetos para a técnica i e w_i é o peso da técnica i .

3.2.3 Seleção de correspondências

Após a *execução do matcher* (e possivelmente da *combinação de resultados*), obtém-se um conjunto de correspondências, ou seja, as técnicas estabelecem as relações que supostamente existem entre os elementos dos dois esquemas de entrada. Entretanto, há a possibilidade de que existam correspondências neste conjunto que não reflitam as relações que realmente existem entre os elementos dos esquemas (falsos positivos). Por isso, na etapa seguinte do processo, deve ser feita a *seleção de correspondências*.

Essa seleção pode ser feita por meio de uma filtragem a partir de um *threshold* específico. Assim, são mantidos apenas os pares de elementos mais similares. Outra abordagem é estabelecer um casamento estável (*stable marriage*), que consiste na atribuição (de um único elemento do primeiro conjunto a um único elemento do segundo) tal que qualquer alteração fornece um resultado pior.

3.3 Considerações finais

Técnicas de matching

Comparar dois esquemas a fim de realizar o *matching* requer uma análise que pode ser decomposta na comparação de suas respectivas entidades (por exemplo: conceitos e propriedades), das instâncias dessas entidades (se estiverem disponíveis) e/ou da própria estrutura dos esquemas (a maneira como as entidades e suas instâncias se dispõem nessa estrutura) (Euzenat e Shvaiko, 2007). Cada tipo de esquema fornece diferentes tipos de entidades para que essa análise seja feita, como conceitos, tabelas, propriedades e atributos.

A escolha da técnica utilizada em uma solução de *matching* depende do tipo de elementos dos esquemas de entrada, e mais especificamente, das características que esses elementos oferecem para análise. Assim, há uma grande variedade de técnicas que podem ser aplicadas para realizar o *matching* entre dois esquemas/ontologias. Tais técnicas provêm de diversas áreas de pesquisa, como Banco de Dados, Recuperação de Informação e Inteligência Artificial.

Considerando as perspectivas dessas áreas, neste capítulo são apresentadas técnicas básicas de *matching*, tomando como base os *surveys* publicados na literatura da área (Rahm e Bernstein (2001), Shvaiko e Euzenat (2005) e Euzenat e Shvaiko (2007)). Na Seção 4.1 é dada uma definição formal de técnica de *matching*, a qual pode ser considerada uma função que, dados dois elementos, retorna o valor que representa a similaridade entre eles.

Em seguida é apresentada uma visão geral das técnicas básicas que podem ser adotadas no processo de *matching* para calcular a similaridade entre os elementos. As técnicas estão distinguidas e organizadas de acordo com os tipos de entrada considerados para análise, que, de acordo com Euzenat e Shvaiko (2007), podem ser: *termos* (Seção 4.2), *estruturas* (Seção 4.3), *instâncias* (Seção 4.4) e *modelos semânticos* (Seção 4.5).

4.1 Técnica: uma função de similaridade

As técnicas de *matching*, que analisam variados tipos de elementos, são blocos de construção para um processo completo e funcional de *matching*. São aplicadas, basicamente, para calcular a *medida de similaridade* entre dois elementos, geralmente normalizada no intervalo $[0, 1]$. Cada uma dessas

técnicas podem ser representadas como uma função (σ), que toma como entrada duas entidades, uma de cada ontologia (O_1 e O_2), e calcula a medida de similaridade entre elas:

$$\sigma : E \times E \rightarrow \mathbb{R}$$

em que $E = E_1 \cup E_2$, sendo E_1 o conjunto de entidades da ontologia O_1 e E_2 o conjunto de ontologias de O_2 . A função de similaridade deve satisfazer algumas propriedades (Euzenat e Shvaiko, 2007):

- positividade: $\forall x \in E, y \in E, \sigma(x, y) \geq 0$
- maximalidade: $\forall x, y, z \in E, \sigma(x, x) \geq \sigma(y, z)$
- simetria: $\forall x, y \in E, \sigma(x, y) = \sigma(y, x)$

Em alguns casos, torna-se mais adequado aplicar a função de distância (δ), uma medida de dissimilaridade entre as entidades. Para a distância, as propriedades que devem ser satisfeitas são:

- positividade: $\forall x \in E, y \in E, \delta(x, y) \geq 0$
- minimalidade: $\forall x \in E, \delta(x, x) = 0$
- simetria: $\forall x, y \in E, \delta(x, y) = \delta(y, x)$
- definibilidade: $\forall x, y \in E, \delta(x, y) = 0$ se e somente se $x = y$
- desigualdade triangular: $\forall x, y, z \in E, \delta(x, y) + \delta(y, z) \geq \delta(x, z)$

Pelas definições dadas, as funções de similaridade e dissimilaridade mapeiam pares de entidades a números reais. Vale ressaltar que a similaridade normalizada ($\bar{\sigma}$) no intervalo $[0, 1]$ corresponde a dissimilaridade normalizada ($\bar{\delta}$) de acordo com a expressão $\bar{\delta} = 1 - \bar{\sigma}$.

4.2 Técnicas baseadas em termos

Ao analisar os componentes de um esquema, pode-se começar pelos nomes que os identificam. Essas técnicas podem ser aplicadas aos identificadores, aos *labels* ou até mesmo aos comentários das entidades a fim de encontrar aquelas que são similares. E, ao extrair os termos referentes aos elementos de um esquema, as técnicas podem interpretá-los de duas maneiras durante o cálculo da similaridade: como uma sequência de caracteres ou como palavras.

4.2.1 Strings

Métodos baseados em *strings* consideram os termos como uma sequência de caracteres. A possibilidade mais simples para determinar a similaridade entre duas *strings* s e t é o **teste de igualdade** entre elas:

$$\sigma(s, t) = \begin{cases} 0 & \text{se } s \neq t \\ 1 & \text{se } s = t \end{cases}$$

Pode-se testar a igualdade após realizar alguma normalização sintática, como transformar todos os caracteres em letras minúsculas (caixa baixa) e eliminação de pontuação. Por exemplo, C.D. se torna cd.

Essa abordagem não considera que as *strings* possam conter erros de digitação e, portanto, caso as *strings* comparadas apresentem algum erro, por menor que seja, o teste de igualdade resulta em 0. Por essa razão, existem métodos que partem do pressuposto de que as *strings* a serem comparadas são sequências errôneas de caracteres.

A **similaridade n -gram** pode ser usada nesse caso. A técnica calcula o número de n -grams, sequências de n caracteres, comuns às duas *strings*. Por exemplo, os trigramas ($n = 3$) para a *string* *article* são: art, rti, tic, icl e cle. Considerando então que $n\text{-gram}(s, n)$ é o conjunto de todas *substrings* de tamanho n que podem ser obtidas da *string* s , a medida de similaridade n -gram para duas *strings* s e t é definida como:

$$\sigma(s, t) = |n\text{gram}(s, n) \cap n\text{gram}(t, n)|$$

em que se calcula um número natural, representando a quantidade de n -grams comuns a s e t . Como as medidas de similaridades são, de preferência, limitadas ao intervalo $[0, 1]$, há também a versão normalizada da função:

$$\bar{\sigma}(s, t) = \frac{|n\text{gram}(s, n) \cap n\text{gram}(t, n)|}{\min(|s|, |t|) - n + 1}$$

Para *strings* curtas, a função n -gram calcula similaridades muito baixas, mesmo se for o caso em que apenas dois caracteres foram trocados, por exemplo. As *strings* *label* e *laebl* apenas têm um bigram ($n = 2$) em comum (*la*) e então a similaridade normalizada bigram dessas duas *strings* é $\frac{1}{4}$. Para lidar com esse problema, *strings* pequenas podem ser estendidas adicionando-se os mesmos caracteres extras no começo e no final de cada *string*.

Outra opção para comparar *strings* é a **distância de edição**, uma medida de dissimilaridade cujo valor obtido para dois objetos representa o custo mínimo de operações a serem aplicadas em um dos objetos a fim de obter o outro. As operações consideradas geralmente são inserção, exclusão e substituição de caracteres. A cada operação é atribuído um custo, que pode variar para cada operação, e a distância entre duas *strings* é calculada como o somatório dos custos de cada operação do conjunto menos custoso de operações.

Assim, dado um conjunto Op de operações em *strings* ($op : \mathbb{S} \rightarrow \mathbb{S}$), e uma função w para atribuir o custo referente à aplicação de cada uma ($w : Op \rightarrow \mathbb{R}$) e considerando que para qualquer par de *strings* há uma sequência de operações que transforma a primeira na segunda (e vice-versa), a distância de edição¹ entre as *strings* s e t é dada por:

$$\delta(s, t) = \min_{(op_i)_{i \in I}; op_n(\dots op_1(s))=t} \left(\sum_{i \in I} w_{op_i} \right)$$

Em alguns casos é útil não apenas comparar *strings* individuais, mas também conjuntos de *strings*.

¹Quando os custos de todas as operações são iguais a 1, a medida é chamada de **distância Levenshten**.

As técnicas a seguir têm origem na área de Recuperação de Informação e consideram uma *string* como um multiconjunto de palavras (*bag of words*), ou seja, um conjunto em que um item particular pode aparecer diversas vezes.

Por funcionarem melhor em textos que abrangem muitas palavras, essa abordagem pode ser adaptada a entidades do esquema da seguinte maneira:

- Agregando diferentes fontes de *strings*: identificadores, *labels*, comentários, documentação, etc.
- Dividindo *strings* em partes independentes. Por exemplo, *peer-reviewed article* se torna *peer*, *reviewed* e *article*.

As entidades do esquema são então transformadas em multiconjuntos, possibilitando que diversas funções de similaridade definidas por técnicas de Recuperação de Informação sejam aplicadas. Ao contrário das técnicas baseadas em *strings* individuais, essas medidas não são mais intrínsecas às *strings* a serem comparadas, mas dependem do conjunto em que as *strings* aparecem.

Em geral, considera-se o multiconjunto s como um vetor $\vec{s}$, pertencente a um espaço vetorial V de n dimensões, em que cada dimensão corresponde a um termo e cada posição do vetor é o número de ocorrências do termo no multiconjunto. No exemplo da Tabela 4.1, podemos observar dois vetores relativos às *strings* s e t . Assim, o vetor formado para cada uma das *strings* são $\vec{s} = (5, 15, 7, 0, 11)$ e $\vec{t} = (7, 18, 2, 8, 0)$.

Tabela 4.1: Exemplo de vetores com as frequências dos termos de cada documento

	“técnica”	“matching”	“ontologia”	“esquema”	“semântica”
s	5	15	7	0	11
t	7	18	2	8	0

Cada documento (fonte do multiconjunto de palavras) pode ser considerado como um ponto nesse espaço e identificado por seu vetor, como uma coordenada (Euzenat e Shvaiko (2007) *apud* Salton (1971) e Salton e McGill (1983)). Uma vez que as entidades foram transformadas em vetores, distâncias métricas comuns podem ser usadas, como a **distância euclidiana**:

$$\delta(\vec{s}, \vec{t}) = \sqrt{\sum_{i=1}^n (\vec{s}_i - \vec{t}_i)^2}$$

Outra medida muito aplicada é a **similaridade do cosseno**, a qual mede o cosseno do ângulo entre os dois vetores. Dados dois vetores $\vec{s}$ e $\vec{t}$, correspondentes a duas *strings* s e t , em um espaço vetorial V de n dimensões, a função correspondente à similaridade do cosseno ($\sigma_V : V \times V \rightarrow [0, 1]$) é dada por:

$$\sigma_V(s, t) = \frac{\sum_{i=1}^n \vec{s}_i \times \vec{t}_i}{\sqrt{\sum_{i=1}^n \vec{s}_i^2 \times \sum_{i=1}^n \vec{t}_i^2}}$$

Considerando que os termos estão organizados em multiconjuntos, é possível avaliar a relevância de um termo específico por meio da medida **TF-IDF** – *Term frequency-Inverse document frequency*

(Euzenat e Shvaiko (2007) *apud* Robertson e Spärck Jones (1976); Manning et al. (2008)). Não se trata de uma medida de similaridade: avalia a relevância de um termo pela comparação da frequência do aparecimento do termo no documento com relação a sua frequência em uma coleção de documentos.

Adota-se essa estratégia de analisar a frequência da palavra na coleção de documentos pois apenas a alta taxa de ocorrência de uma palavra em um documento isolado pode não ser suficiente para essa avaliação. Nem todos os termos podem ser considerados igualmente para avaliar qual conteúdo é relevante ao documento. Por exemplo, uma coleção de documentos da indústria automobilística provavelmente apresenta a palavra **automóvel** em quase todos os documentos. Portanto, tal palavra não diferencia o conteúdo relevante entre os documentos.

A ideia da técnica TF-IDF é combinar uma alta frequência de um termo em um documento específico ($tf_{t,d}$) com uma baixa frequência de documentos da coleção que contenham o termo (idf_t), para poder assim indicar alta relevância do termo.

A princípio, é preciso diminuir o impacto de termos que tenham alta frequência na coleção. A *frequência inversa dos documentos* considera o número de documentos na coleção que contêm o termo t (df_t) e o relaciona ao número total de documentos na coleção (N):

$$idf_t = \log \frac{N}{df_t}$$

Em seguida, é possível combinar a frequência do termo com a frequência inversa de documentos:

$$tf-idf_{t,d} = tf_{t,d} \times idf_t$$

A medida $tf-idf_{t,d}$ atribui a um termo t um peso em relação ao documento d que é:

- mais alto quando t ocorre muitas vezes dentro de um número pequeno de documentos, mas ocorre em poucos documentos na coleção como um todo (o que distingue os documentos dentro da coleção);
- baixo quando o termo ocorre poucas vezes em um documento e/ou ocorre em muitos documentos (o que diminui o sinal de relevância);
- mais baixo quando o termo ocorre em todos os documentos.

4.2.2 Palavras

Até então as *strings* foram consideradas como sequências de caracteres. Entretanto, *strings* também podem ser consideradas textos em alguma linguagem. E os textos, por sua vez, podem ser segmentados em palavras que correspondem a entradas em um dicionário.

Os métodos que consideram *strings* como palavras, os *métodos baseados em linguagem*, têm apoio em técnicas de Processamento de Linguagem Natural para ajudar a extrair os termos de um texto. Comparar esses termos e suas relações ajuda a avaliar a similaridade das entidades da ontologia que eles nomeiam ou comentam.

A fim de considerá-los dessa maneira, é preciso lidar com a possibilidade de ocorrer diferentes formas de *strings* mesmo que representem os mesmos termos, como por exemplo: *theory papers*, *theoretical paper* e *theoretical papers*. Por essa razão, a primeira técnica que deve ser aplicada é a **normalização linguística**. É preciso reduzir um termo a alguma forma padronizada que seja facilmente reconhecida.

Em geral, o conjunto de passos seguidos para a normalização é composto por *tokenização*, *lematização*, *eliminação de stopwords*. A tokenização consiste em separar uma *string* em uma sequência de *tokens* reconhecendo pontuação, espaços e dígitos. Por exemplo, *peer-reviewed periodic publication* se torna o conjunto de tokens `<peer,reviewed,periodic,publication>`.

A lematização consiste em transformar os termos em seus radicais, por exemplo *reviewed* se torna *review*. Por fim, devem ser eliminadas as palavras que não apresentam significado adicional ao texto (não sendo, portanto, significativas ao *matching*), mas ainda assim aparecem com muita frequência, como artigos e preposições (palavras como *to*, *of* ou *a*), as chamadas *stop words*. Assim, *collection of articles* se torna *collection articles*.

Com os termos normalizados, é possível realizar **consultas a recursos linguísticos externos**. Esses recursos, como dicionários e tesouros, podem ser explorados com a finalidade de encontrar similaridades entre os termos. Um *dicionário* contém entradas com uma definição em linguagem natural dessas palavras. Um *tesouro* é um tipo de dicionário ao qual alguma informação relacional foi adicionada. As relações tipicamente representadas por um tesouro são: sinônimo (*paper* significa o mesmo que *article*), hiperônimo (*biography* é mais geral que *autobiography*), hipônimo (*autobiography* é mais específico que *biography*) e antônimo (*practice* é o oposto de *theory*).

Um exemplo de tesouro da língua inglesa é o WordNet (Miller (1995), Fellbaum (1998)), que agrupa as acepções das palavras em grupos de sinônimos (*synsets*). O uso mais simples dos *synsets* é a **similaridade dos sinônimos**, em que dados dois termos s e t e um recurso do qual se obtém os sinônimos (Σ), é uma função $\sigma : \mathbb{S} \times \mathbb{S} \rightarrow [0, 1]$ definida por:

$$\sigma(s, t) = \begin{cases} 1 & \text{se } \Sigma(s) \cap \Sigma(t) \neq \emptyset \\ 0 & \text{caso contrário} \end{cases}$$

em que, os termos são considerados similares apenas se possuem algum sinônimo em comum. Por outro lado, a **similaridade dos co-sinônimos** ($\sigma : \mathbb{S} \times \mathbb{S} \rightarrow [0, 1]$), definida por:

$$\sigma(s, t) = \frac{|\Sigma(s) \cap \Sigma(t)|}{|\Sigma(s) \cup \Sigma(t)|}$$

ou seja, permite que os termos sejam comparados de acordo com o número de sinônimos que têm em comum.

Outro recurso externo que pode ser explorado é uma *terminologia*. Terminologias são vocabulários de domínio específico e tendem a ser menos ambíguos do que dicionários. As entradas de uma terminologia são mais adequadas quando os textos ou esquemas a serem casados dizem respeito ao mesmo domínio, por reterem acepções mais especializadas ou até mesmo acepções que não existem na linguagem cotidiana.

Em suma, os recursos externos linguísticos podem ser definidos para uma linguagem e/ou serem específicos a algum domínio. Podem conter também abreviações comuns usadas no domínio. Por exemplo, uma companhia pode expandir PO como Purchase Order em vez de Post Office ou Project Officer.

4.3 Técnicas baseadas na estrutura

Adicionalmente à comparação de nomes das entidades de um esquema, as suas estruturas também podem ser comparadas. A estrutura das entidades que um esquema fornece pode ser analisada de duas maneiras. Pode ser analisada a definição isolada (estrutura interna) de uma entidade do esquema (atributos, identificadores, tipos de dados) ou pode-se considerar as relações mantidas com as outras entidades do esquema (estrutura relacional). A estrutura interna é mais explorada em *matching* de esquemas de banco de dados, enquanto a estrutura relacional é mais importante no contexto de *matching* de ontologias.

4.3.1 Estrutura interna de uma entidade

Esquemas contêm restrições para definir cada entidade, como tipos de dados dos atributos (ou propriedades) e cardinalidades. As técnicas baseadas na estrutura interna das entidades calculam a similaridade com base nessas restrições.

A primeira técnica que pode ser aplicada é a **comparação de tipos de dados**. Cada atributo (ou propriedade) de uma entidade é restringido por um tipo de dados, e entidades similares provavelmente apresentam atributos com tipos de dados similares. Portanto, além de buscar por correspondência exatas de tipos, é preciso determinar o quanto dois tipos de dados parecidos são similares. Por exemplo, os tipos `integer` e `short` não podem ser considerados completamente incompatíveis.

O ideal é que a proximidade entre tipos de dados seja máxima quando são os mesmos, mais baixa quando os tipos são compatíveis (seria o caso dos tipos `integer` e `short`) e mínima quando não são compatíveis. A compatibilidade pode ser avaliada com o auxílio de um recurso como uma tabela de análise da compatibilidade dos tipos de dados. A Tabela 4.2 representa algumas entradas de um exemplo dessa tabela.

Tabela 4.2: Parte de uma tabela de compatibilidade de tipos de dados (Euzenat e Shvaiko, 2007)

	char	fixed	enumeration	int	number	string
string	0.7	0.4	0.7	0.4	0.5	1.0
number	0.6	0.9	0.0	0.9	1.0	0.5

Outra técnica que pode ser aplicada para analisar a estrutura interna de uma entidade é a **comparação da cardinalidade** dos valores de uma propriedade. O procedimento é similar ao da comparação de tipos de dados, e a similaridade pode ser obtida com base em uma tabela como a 4.3.

Focando em esquemas de banco de dados, a **comparação de chaves** é muito útil para reconhecer que duas entidades são as mesmas. A chave, além de identificar as instâncias, também podem ser usadas para identificar classes. Duas classes identificadas da mesma maneira provavelmente representam o mesmo conjunto de objetos. Se uma tabela `Book` tem como chave primária um atributo `isbn` e outra tabela `Volume` também, é um forte indício de que as tabelas se referem ao mesmo conceito.

Tabela 4.3: Uma tabela de compatibilidade de cardinalidade (Euzenat e Shvaiko, 2007)

	*	+	?	nenhuma
*	1.0	0.9	0.7	0.7
+	0.9	1.0	0.7	0.7
?	0.7	0.7	1.0	0.8
nenhuma	0.7	0.7	0.8	1.0

Tabela 4.4: Exemplo de estrutura interna de entidades a serem casadas (Rahm e Bernstein, 2001)

S1 elements	S2 elements
Employee	Personnel
EmpNo - int, primary key	Pno - int, unique
EmpName - varchar(50)	Pname - string
DeptNo - int, references Department	Dept - string
Salary - dec(15,2)	Born - date
Birthdate - date	
Department	
DeptNo - int, primary key	
DeptName - varchar(40)	

Todos esses métodos são geralmente combinados com outras técnicas, como as baseadas em termos, por ajudarem a reduzir o número de correspondências candidatas. Na Tabela 4.4 são apresentados dois esquemas de banco de dados que devem ser casados com técnicas de estrutura interna. As informações sobre tipos de dados e sobre chaves sugerem que Born casa com Birthdate e Pno casa com EmpNo ou DeptNo. Os elementos restantes do esquema S2 Pname e Dept são *strings*, o que indica um provável *match* com EmpName ou DeptName.

Como o exemplo ilustra, o *matching* que se baseia apenas em informações da estrutura interna geralmente resulta em grupos de candidatos à correspondência, uma vez que existem vários elementos no esquema com restrições comparáveis, como por exemplo atributos de um mesmo tipo de dado. Por isso, devem ser usadas como um passo para eliminar as restrições que são claramente incompatíveis.

4.3.2 Estrutura relacional entre entidades

Quanto à estrutura relacional, uma ontologia pode ser considerada como um *grafo* cujos vértices e arestas representam entidades e as relações entre elas, respectivamente. Para encontrar as correspondências, é preciso então fazer a **busca de subgrafos comuns** entre as duas ontologias. Por causa da complexidade de se realizar essa comparação, o problema é codificado com um problema de otimização, em que se procura subgrafos comuns minimizando alguma distância entre os objetos a serem casados (ou maximizando a similaridade).

Ao se avaliar o grafo formado pelas entidades da ontologia, considerando que as propriedades são arestas do grafo, pode-se também fazer o *matching* a partir da **similaridade baseada nas relações**. Se o conjunto de propriedades (arestas) que partem de duas entidades (vértices) são similares, então é muito provável que as entidades também sejam similares. Por exemplo, se a classe **Book** é relacionada à classe **Human** pela propriedade **author** em uma ontologia e a classe **Volume** é relacionada à classe

Writer pela propriedade *author*, é possível inferir que Human e Writer podem ser similares.

Por outro lado, o grafo de entidades pode ser formado apenas por relações de subclasse, representando a estrutura taxonômica ou *hierarquia de classes*, que consiste na principal estrutura de uma ontologia. Há diversas medidas propostas para comparar classes com base na estrutura taxonômica. A mais comum é a **dissimilaridade estrutural da hierarquia** dada por:

$$\forall e, e' \in o, \delta(e, e') = \min_{c \in o} [\delta(e, c) + \delta(e', c)]$$

em que $\delta(e, c)$ é o número de arestas intermediárias entre um elemento e e outro elemento c . Portanto a medida expressa a quantidade de arestas na hierarquia entre os elementos e e e' .

Ainda com respeito a analisar hierarquias, as relações de super ou subclasse também servem para analisar a similaridade. A ideia é que se classes são similares então suas super ou subclasses também o são. Essa abordagem se torna um problema pois, ao analisar a similaridade de uma classe, é necessário calcular a similaridade de suas sub ou superclasses, que por sua vez também se apóiam na similaridade de suas sub ou superclasses, o que se transforma em um problema global.

4.4 Técnicas baseadas em instâncias

Há casos em que as informações relativas ao esquema são limitadas ou em que não são fornecidas junto aos dados. Não obstante, as instâncias podem indicar aspectos do conteúdo e do significado de elementos do esquema subjacente. E, mesmo quando o esquema esteja disponível, a análise das instâncias pode ajudar a escolher a correspondência mais adequada quando, ao aplicar alguma técnica de *matching*, se obtém um conjunto de opções plausíveis.

Se dois conceitos (ou classes) compartilham exatamente o mesmo conjunto de instâncias, pode-se presumir que essas classes são similares. Por outro lado, mesmo quando as classes não compartilham o mesmo conjunto de instâncias, o processo do *matching* pode ser fundamentado em aspectos das instâncias que não mudam. As **técnicas de identificação de instâncias** se aplicam nessa situação, pois tentam identificar as instâncias de um conjunto que correspondem às instâncias do segundo conjunto. Por exemplo, se dois valores da propriedade *title* de instâncias pertencentes ao conceito *Book* forem diferentes, então muito provavelmente não se referem ao mesmo livro.

Para casar duas classes de um esquema, o conjunto de instâncias de cada uma deve ser ao menos similar. Portanto, as técnicas baseadas em instâncias precisam de alguma maneira avaliar os conjuntos relativos a cada par de classes. A granularidade dessa análise varia. Podem ser analisados conjuntos de instâncias como um todo ou podem ser analisados pares de instâncias relativos a cada conjunto. No segundo caso, avalia-se a similaridade dos conjuntos com base na similaridade das instâncias contidas, permitindo que dois conjuntos que não tenham instâncias idênticas em comum, mas que apresentem instâncias similares, possam ser considerados conjuntos similares também.

Quando as ontologias a serem casadas compartilham o mesmo conjunto de instâncias, pode-se aplicar técnicas de **comparação de conjuntos comuns** a fim de estabelecer as correspondências dos elementos. A maneira mais simples de comparar duas classes que compartilham instâncias é testar a intersecção dos seus respectivos conjuntos de instâncias A e B e considerar que essas classes são muito similares quando $A \cap B = A = B$ ou que uma é mais geral do que a outra quando $A \cap B = A$

ou $A \cap B = B$. O problema com essa abordagem é a habilidade de lidar com falhas: por exemplo, se as classes têm algumas instâncias em comum, mas não todas, a dissimilaridade já resultaria no valor máximo.

Uma maneira de resolver esse problema é usar a **distância de Hamming**, que calcula o tamanho da diferença simétrica entre os dois conjuntos (a quantidade de instâncias que não são comuns a ambos) e normaliza o valor com o tamanho da união. Usando-a para comparar conjuntos permite a tolerância de algumas instâncias classificadas erroneamente e ainda assim produz uma distância pequena. A distância de Hamming entre dois conjuntos é a função de dissimilaridade $\delta : 2^I \times 2^I \rightarrow \mathbb{R}$ tal que $\forall X, Y \subseteq I$:

$$\delta(X, Y) = \frac{|X \cup Y - X \cap Y|}{|X \cup Y|}$$

Uma questão que é válido ressaltar sobre as abordagens acima é que apenas as instâncias idênticas são consideradas na intersecção entre os conjuntos. Instâncias similares não são levadas em consideração, e, por isso, o cálculo da similaridade é mínimo quando as ontologias não compartilham instâncias. Nesse caso deve-se aplicar a **comparação de conjuntos disjuntos**, calculando a similaridade do conjunto de instâncias com base na similaridade entre os elementos individuais.

E, para comparar os conjuntos com base na similaridade dos elementos pode-se usar o **método do acoplamento médio**, por exemplo, que permite avaliar a distância entre dois conjuntos cujos objetos são apenas similares, e não necessariamente idênticos. Dada uma função de dissimilaridade para avaliar as instâncias $\delta : I \times I \rightarrow \mathbb{R}$ (que pode ser proveniente de qualquer outra técnica básica adequada), a medida do acoplamento médio entre dois conjuntos é uma outra função de dissimilaridade $\Delta : 2^I \times 2^I \rightarrow \mathbb{R}$ tal que $\forall X, Y \subseteq I$:

$$\Delta(X, Y) = \frac{\sum_{(e, e') \in X \times Y} \delta(e, e')}{|X| \times |Y|}$$

4.5 Técnicas baseadas em um modelo semântico

A característica chave de métodos semânticos é serem dedutivos, uma vez que a justificação de seus resultados é dada por meio de semântica formal. Entretanto a natureza do raciocínio do processo de *matching* é, em essência, indutiva, pois pela análise das informações específicas disponíveis (sejam termos, estruturas ou instâncias) chega-se a uma conclusão generalizada, como, por exemplo, que uma classe seja equivalente à outra.

Portanto, os métodos baseados em semântica precisam de uma fase de pré-processamento que proveja *âncoras*, alinhamentos semente que sirvam de entrada aos métodos semânticos. Por exemplo, o usuário do sistema de *matching* pode declarar que entidades das duas ontologias a serem casadas são equivalentes ou essa relação de equivalência pode ter sido estabelecida pelo próprio sistema por meio da análise dos nomes das entidades (Subseção 4.2.1). Os métodos semânticos servem como amplificadores dessas âncoras providas como entrada.

Quando duas ontologias devem ser casadas, elas geralmente carecem de uma base comum em que comparações semânticas podem ser feitas. Assim, a utilização de uma **ontologia externa**, que seja

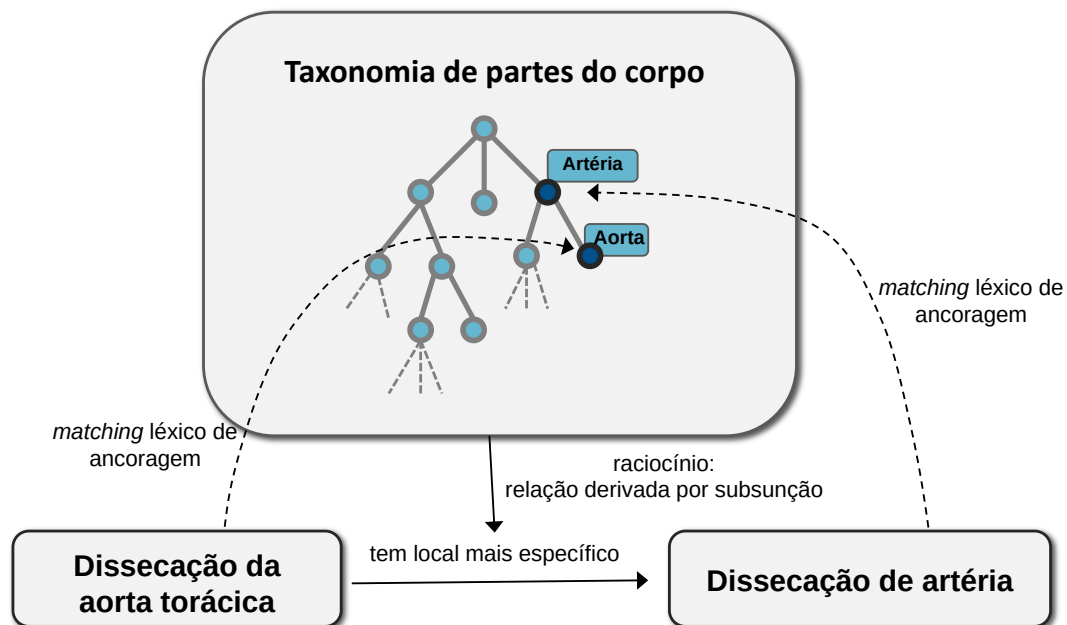


Figura 4.1: *Matching* semântico com o auxílio de alinhamentos âncora (Aleksovski et al., 2006)

intermediária entre as duas, pode colaborar em estabelecer as âncoras que são necessárias para iniciar o *matching*. Outros tipos de recursos também podem ser aplicados na ancoragem, como tesauros, e até mesmo as instâncias das ontologias.

Para estabelecer as âncoras de maneira automática, a ontologia externa (ou outro tipo de recurso) é casada individualmente com cada ontologia de entrada do processo de *matching* e são produzidos alinhamentos âncora. A ideia é que uma ontologia externa que apresenta uma cobertura abrangente do domínio de interesse das duas outras ontologias ajuda a eliminar a ambiguidade do significado dos seus termos.

Quando dois conceitos A e B são comparados com base em âncoras A' e B' , respectivamente, pode-se comparar A' e B' , e, caso sejam relacionados, é possível inferir que A e B também o são. Como pode ser visto no exemplo da Figura 4.1, o conceito Dissecação de artéria tem o local Artéria (de acordo com as propriedades declaradas na ontologia), e o conceito Dissecação da aorta torácica tem o local Aorta torácica. Uma vez que descrevem partes anatômicas relacionadas, a relação entre esses dois conceitos médicos é inferida: de acordo com a ontologia intermediária, Aorta é um tipo de Artéria. Então o conceito Dissecação da aorta torácica pode ser inferido como tendo um local mais específico do que o conceito Dissecação de artéria (Aleksovski et al., 2006).

Os alinhamentos iniciais são obtidos por qualquer outro método, geralmente aplica-se os que são mais simples. Ainda na Figura 4.1, pode ser considerado que os conceitos Aorta e Artéria foram casados por meio de técnicas léxicas, ou seja, por técnicas baseadas em *strings* (Subseção 4.2.1). Uma vez que esses alinhamentos iniciais tenham sido obtidos, podem ser explorados em seguida por **técnicas dedutivas**.

A base das técnicas semânticas é a fusão (*merging*) das duas ontologias o e o' e a busca por correspondências A tais que $o, o' \models A$, sendo que A é uma fórmula da linguagem na qual o e o' estão expressas: por exemplo, se o alinhamento expressa uma relação de subsunção entre e e e' , então

$e \subseteq e'$.

O propósito mais importante do uso das técnicas semânticas é o teste de satisfatibilidade dos alinhamentos encontrados por outras técnicas, ou seja, verificar se a fusão das ontologias se torna inconsistente com a adição do alinhamento. Caso fique, esse alinhamento deve ser considerado um falso positivo e descartado. Entre as técnicas que podem ser aplicadas, as mais conhecidas são a *satisfatibilidade proposicional* e as *técnicas baseadas em lógica descritiva*.

Uma abordagem para aplicar **técnicas de satisfatibilidade proposicional** (SAT), inclui os passos:

1. Para as duas ontologias de entrada, construir uma teoria² *Axiomas* como uma conjunção dos axiomas disponíveis.
2. Construir uma fórmula de *matching* para cada par de classes c e c' das duas ontologias. O critério para determinar se existe uma relação entre as duas classes é o fato de que seja decorrente das premissas (a teoria):

$$Axiomas \rightarrow r(c, c')$$

para cada par de classes c e c' para os quais se deseja testar a relação r , sendo que r pode ser $=, \sqsubseteq, \sqsupseteq$ e $\perp$.

3. Verificar a validade da fórmula, ou seja, confirmar que é verdadeira para todas as atribuições de todas as suas variáveis proposicionais. Uma fórmula proposicional é válida se e somente se a sua negação é insatisfatível, o que se verifica por meio de um SAT *solver*. SAT *solvers* são sistemas de decisão para satisfatibilidade proposicional, e portanto, podem ser usados para uma verificação completa de todas as correspondências possíveis.

A técnica acima é aplicada em sentenças no formato da lógica proposicional. Por outro lado, há também a abordagem de se aplicar **técnicas de lógica descritiva**. Na lógica descritiva todas as relações, por exemplo $=, \sqsubseteq, \sqsupseteq$ e $\perp$, podem ser expressas com respeito à subsunção.

A subsunção determina relações subclasse-superclasse: C é subsumida por D se e somente se todas as instâncias de C são necessariamente instâncias de D , ou seja, a primeira classe é sempre interpretada como um subconjunto da segunda (Baader et al., 2009).

O teste de subsunção pode ser usado para estabelecer as relações entre as classes em uma maneira puramente semântica. Fazendo primeiro a fusão de duas ontologias e então testando cada par de conceitos por subsunção é suficiente para casar conceitos com a mesma interpretação. Como um exemplo de aplicação da técnica, sejam consideradas duas ontologias mínimas em lógica descritiva:

$$\text{Microempresa} = \text{Empresa} \sqcap \leq_5 \text{ empregado}$$

o que significa que Microempresa é uma Empresa com no máximo 5 empregados e

$$\text{PME} = \text{Firma} \sqcap \leq_{10} \text{ associado}$$

²Conjunto de sentenças

ou seja, PME é uma Firma com no máximo 10 associados. Intuitivamente é possível concluir que ambos os conceitos são relacionados, mas o sistema de *matching* não consegue relacionar os conceitos, considerando apenas essas duas sentenças. Entretanto, é possível fazer a ancoragem dos conceitos para contornar esse problema. Os termos que identificam os conceitos podem ser analisados por técnicas baseadas em termos, por exemplo. Outra abordagem é permitir que o usuário forneça o alinhamento inicial de entrada, como:

$$\begin{aligned} \text{Empresa} &= \text{Firma} \\ \text{associado} &\sqsubseteq \text{empregado} \end{aligned}$$

que expressa que Empresa é equivalente a Firma e que associado é uma subclasse de empregado. Isso implica que:

$$\text{Microempresa} \sqsubseteq \text{PME}$$

isto é, que Microempresa é uma subclasse de PME.

4.6 Considerações finais

O processo de *matching* toma como entrada dois esquemas e retorna um conjunto de correspondências. Tais correspondências declaram quais elementos dos esquemas são equivalentes (ou outra relação que houver entre eles).

As técnicas utilizadas para compará-los se baseiam em alguma característica específica desses elementos para poder decidir se são similares ou não. Por exemplo, no caso em que se avalia dois conceitos de esquemas distintos, é possível analisá-los, entre outras características, com base nos identificadores, nos tipos de dados dos atributos ou no conjunto de instâncias de cada um.

Portanto, de acordo com os elementos disponibilizados pelos esquemas, devem ser escolhidas técnicas adequadas para avaliar a similaridade entre pares de elementos. Neste capítulo foram apresentadas técnicas básicas, organizadas de acordo com os argumentos providos como entrada (*termos*, *conceitos*, *instâncias* e *teorias lógicas*), como esquematizado na Figura 4.2.

Para cada tipo de entrada foram apontados exemplos de técnicas que podem ser aplicadas, de acordo com Euzenat e Shvaiko (2007). Na Tabela 4.5 estão listadas as técnicas citadas ao longo do capítulo. Para escolher quais técnicas devem compor a fase de execução do processo de *matching* é preciso analisar quais tipos de elementos os esquemas de entrada podem oferecer, para então selecionar as técnicas que conseguem calcular a similaridade entre esses elementos.

Como pôde ser observado, cada técnica básica avalia e calcula a similaridade entre um par de elementos com base em apenas um aspecto para supor uma relação de equivalência entre eles. Portanto, as técnicas não podem ser usadas isoladamente, mas cada uma delas pode se beneficiar dos resultados providos por outras (Euzenat e Shvaiko, 2007). A aplicação de várias técnicas permite que vários aspectos dos elementos sejam considerados ao estabelecer uma correspondência. Portanto, é preciso escolher as técnicas a serem implementadas pelos *matchers* individuais do processo e os compor da maneira mais adequada para a fase de execução do processo de *matching* (vide Figura 3.3).

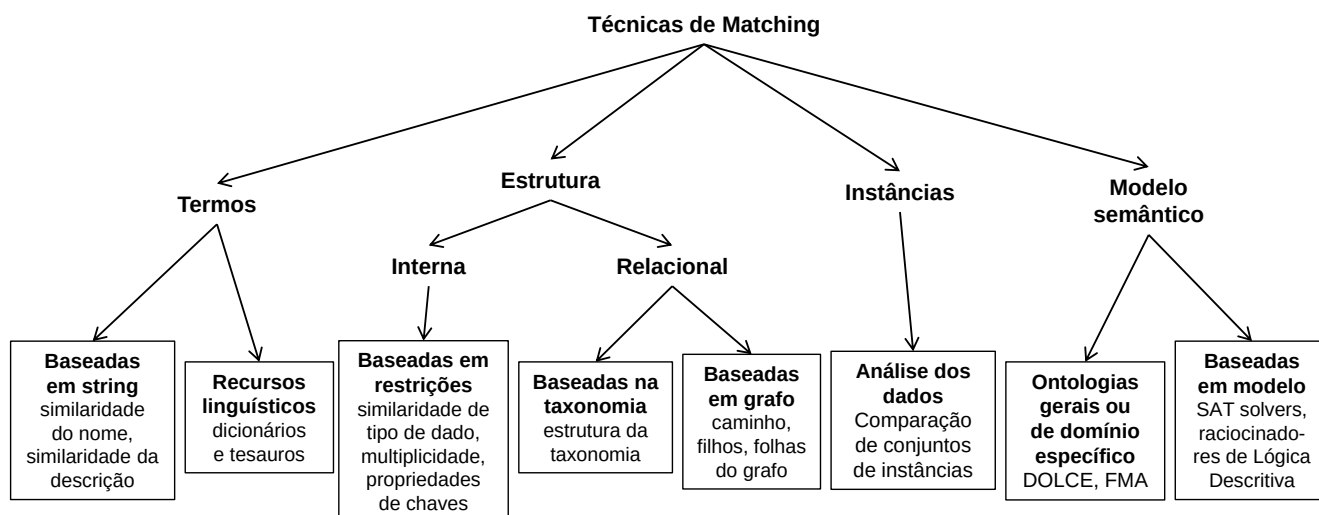


Figura 4.2: Classificação das técnicas de *matching* de acordo com o tipo de entrada (adaptada de Euzenat e Shvaiko (2007))

Tabela 4.5: Técnicas básicas de *matching*

strings	igualdade de strings
	similaridade n-gram
	distância de edição
conjuntos de strings (bags of words)	similaridade do cosseno
	TFIDF
palavras	normalização linguística
	consulta a recursos linguísticos externos (dicionários/tesouros/terminologias)
	similaridade de sinônimos
	similaridade de co-sinônimos
estrutura interna	comparação de chaves (BD)
	comparação de tipo de dado
	comparação de multiplicidade
estrutura relacional	busca de subgrafos comuns
	dissimilaridade estrutural da hierarquia
instâncias	comparação entre conjuntos comuns (distância entre os conjuntos)
	identificação de instâncias
	comparação de conjuntos disjuntos
semântica	uso de ontologias externas (ancoragem)
	técnicas dedutivas

Projeto de Pesquisa

Com o propósito de aplicar a ontologia no empacotamento de experimentos controlados em ES, inicialmente foi proposta a instanciação das informações de pacotes de laboratório baseada nos conceitos da *EXPEROntology*, de acordo com o processo de experimentação (Wohlin et al. (2000); Amaral e Travassos (2003)). Para apoiar essa abordagem inicial, a *EXPEROntology* foi implementada em OWL e, então, uma ferramenta foi desenvolvida para instanciar os conceitos, a PontoLab (Scatalon et al., 2011b).

Entretanto, durante a validação da ferramenta, foi enfrentado o problema da falta de padronização, o que ocasiona dificuldades na integração de conhecimento gerado por cada estudo isolado, assim como foi exposto na Seção 2.4. Assim, não apenas o uso da *EXPEROntology* para empacotar o conjunto de informações de experimentos deve ser considerado, mas também a prática de publicação atual, uma vez que é comum haver pacotes de laboratório contendo diferentes conjuntos de informação. Portanto, é necessário lidar com diferentes conjuntos de informação para acomodá-los de acordo com a *EXPEROntology*, que pode ser utilizada para integrar as diferentes perspectivas sobre o que deve conter um pacote de laboratório.

Uma vez que há a necessidade de uma padronização que considere a irregularidade dos conjuntos de informação dos experimentos publicados (Carver, 2010), a *EXPEROntology* deve lidar com as diferentes maneiras que a execução de um experimento pode ser registrada. Isso sugere que a ontologia inicial deve evoluir, incorporando novos conceitos.

5.1 Objetivos

Nessa direção, como primeira etapa deste projeto de mestrado foi proposto um *workflow* para empacotamento de estudos (Scatalon et al., 2011a), que inclui uma abordagem evolutiva para a instanciação de pacotes de laboratório usando a *EXPEROntology*. Cada fase do processo de experimentação (Wohlin et al., 2000) tem sua própria meta e seus respectivos conceitos definidos na ontologia. Seguindo o processo, os conceitos são usados como parâmetros para equiparar (*match*) conceitos da ontologia correspondentes às informações não padronizadas. Desse modo, é possível instanciar

informação em um pacote de laboratório padronizado de acordo com a ontologia.

Como ilustrado na Figura 5.1, as entradas do *workflow* são o pacote de laboratório a ser instanciado e a *EXPEROntology*. A primeira atividade do *workflow* se repete para cada fase do processo de experimentação: deve ser feita a seleção de informações do pacote de laboratório correspondentes a cada fase. Em seguida, dados os conceitos definidos na *EXPEROntology* e as informações selecionadas para uma fase específica do processo, equipara-se (*match*) conceitos e informações. A atividade de *matching* visa a identificar (i) as informações às quais a *EXPEROntology* pode fornecer conceitos definidos para instanciiação e (ii) as informações sem conceito correspondente. Tais informações, portanto, indicam os conceitos que devem ser incorporados à ontologia (na atividade de evolução) para possibilitar a instanciiação completa, gerando um pacote de laboratório padronizado.

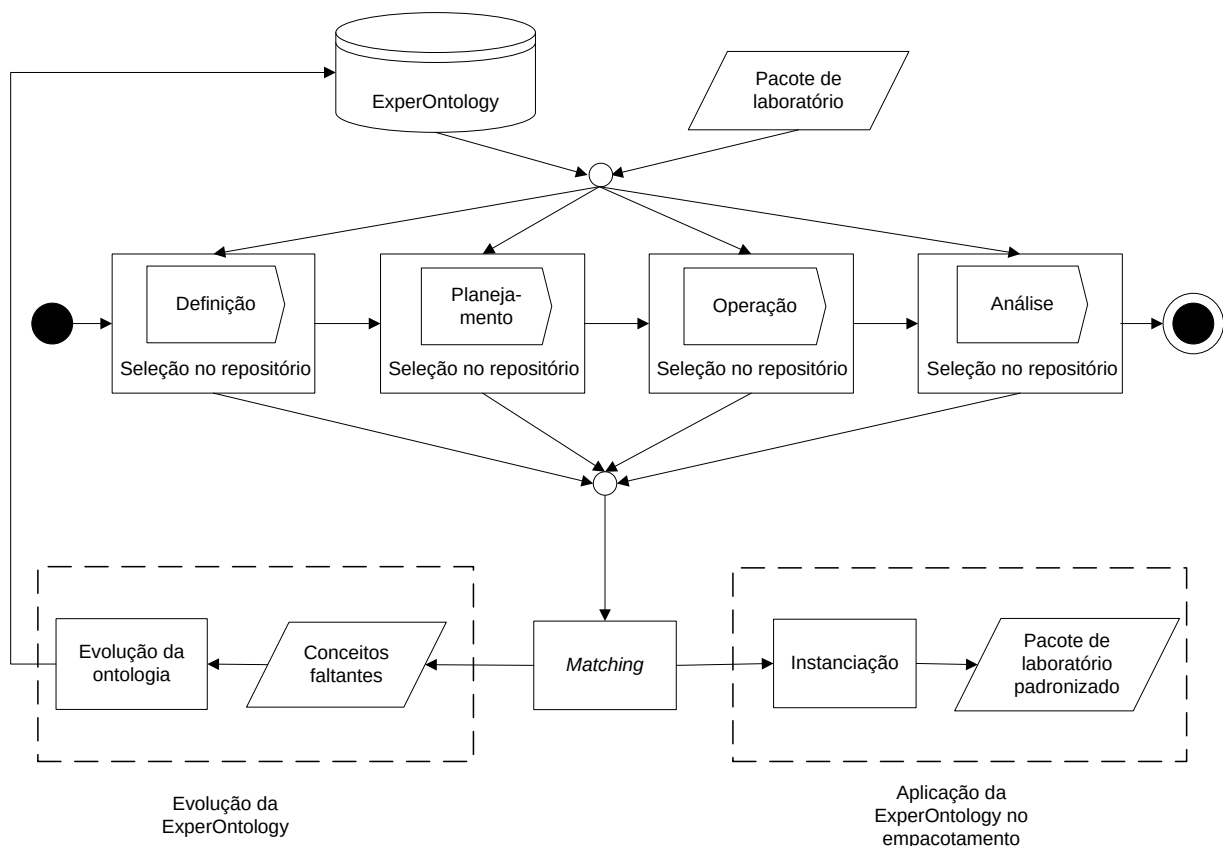


Figura 5.1: *Workflow* para empacotar experimentos controlados (Scatalon et al., 2011a)

Dessa maneira, a execução iterativa do *workflow* caracteriza um processo de evolução da ontologia, que se aproxima de um padrão semântico do conjunto de informações que um pacote de laboratório deve conter. Considerando que a atividade de instanciiação foi implementada na ferramenta PontoLab, para operacionalizar o *workflow* é preciso implementar as atividades de seleção, de *matching* e de evolução da ontologia.

5.2 Justificativa

A maneira que o pacote de laboratório fica armazenado na ontologia facilita a transferência do conhecimento nele contido. Por se tratar de uma especificação formal, é possível compartilhar um

entendimento comum da estrutura da informação entre os diferentes grupos de pesquisa (Noy e McGuinness, 2001). Dessa maneira, a ontologia de instâncias de um pacote de laboratório gerado por um determinado grupo de pesquisa, pode ser mais facilmente processado em outro grupo, de um contexto diferente do grupo de origem. Isso é possível pois a ontologia embute a semântica do domínio em que está inserida, ou seja, contém o significado das informações representadas. Os pacotes de laboratório não seriam tão portáteis se fossem instanciados em bancos de dados relacionais, por exemplo.

No contexto do FIRE, a atividade *criar/evoluir pacote* sugere que a ontologia deve evoluir, considerando que a *EXPEROntology* é tomada como modelo para o pacote de laboratório. A proposta deste trabalho também apóia as atividades do ciclo externo do FIRE, pois busca um padrão que acomode os diferentes conjuntos de informação visando à integração do conhecimento de um estudo isolado em um corpo comum. E, conforme pacotes de laboratório são instanciados usando o *workflow* proposto, a *EXPEROntology* se aproxima de um padrão semântico que permite acomodar variações de pacotes de laboratório. Desse modo, a evolução da *EXPEROntology* também influencia na sua aplicação em empacotar experimentos.

5.3 Metodologia

Este projeto de mestrado tem como meta contribuir na integração de estudos isolados em um corpo comum com base na padronização por meio da *EXPEROntology*. Portanto, o *workflow* proposto deve considerar os diversos formatos em que o pacote de laboratório de entrada pode estar representado. Entre os formatos possíveis, pode-se citar: os estruturados (tais como bases de dados), semi-estruturados (XML, por exemplo) assim como documentos não estruturados (artigos da literatura que descrevem as informações em linguagem natural).

Para projetar uma solução geral ao problema da heterogeneidade de pacotes de laboratório, é preciso que essa variação de formatos em que podem ser representados seja considerada. Assim, caracteriza-se a necessidade de fazer o tratamento dos dados no pacote de laboratório de entrada. Ou seja, a atividade de *matching* do *workflow* deve receber como entrada, além da ontologia, as informações do pacote de laboratório sob uma estrutura que permita realizar o *matching*.

Uma vez que as entradas da atividade de *matching* sejam dois esquemas (a *EXPEROntology* e o pacote de laboratório estruturado), é possível aplicar técnicas de *matching* de esquemas (Rahm e Bernstein (2001); Euzenat e Shvaiko (2007)). Na sequência de passos da Figura ?? para realizar o *matching*, pode-se verificar que deve haver um pré-processamento dos esquemas de entrada, para em seguida servirem de entrada para a execução de um *matcher*, o sistema que de fato realiza o *matching*. Como pode ser visto na Figura 3.3(b), tal sistema consiste em uma composição de diversas técnicas de *matching*, cujos resultados são combinados a fim de encontrar as correspondências entre os esquemas.

A escolha das técnicas que devem compor o *matcher* dependem do formato em que os esquemas estão. Assim, é preciso estabelecer um formato para o pacote de laboratório de entrada a fim de determinar quais técnicas devem ser aplicadas. Portanto, mesmo que não seja padronizado, o pacote de laboratório de entrada deve ser estruturado, de modo a fornecer como entrada ao *workflow* da Figura 5.1 uma estrutura que possa ser equiparada à *EXPEROntology*.

Para gerar essa estrutura comum (a partir de diversos formatos diferentes) é necessário buscar

padrões nas informações a fim de compor tal estrutura. Nesse contexto, caracteriza-se um problema análogo ao que a área de descoberta de conhecimento ou KDD (*Knowledge Discovery in Databases*) lida: mapear dados de mais baixo nível a outros formatos mais compactos e abstratos, como por exemplo, uma aproximação ou um modelo do processo que gerou os dados (Fayyad et al., 1996a).

A descoberta de conhecimento foca em descobrir padrões compreensíveis que possam ser interpretados como conhecimento útil (Fayyad et al., 1996b). No núcleo do processo de KDD, ilustrado na Figura 5.2, está a aplicação de métodos de mineração de dados para descoberta e extração de padrões presentes nos dados (Fayyad et al., 1996b). Assim, o processo de KDD deve ser aplicado para produzir o pacote de laboratório não padronizado em uma estrutura comum, que dessa maneira viabiliza a execução do *workflow* de empacotamento. Por outro lado, para que processo de KDD se adapte a esse domínio de aplicação (que não é o domínio para o qual foi projetado: banco de dados), o passo do processo em que se extraem padrões dos dados por meio de técnicas de mineração de dados deve ser adaptado.

Considerando que se deve extrair padrões de informação de documentos semi-estruturados ou até mesmo em linguagem natural, técnicas de aprendizagem de ontologia (Maedche e Staab, 2004) podem ser aplicadas. De fato, aprendizagem de ontologia não difere substancialmente de mineração de dados (Cimiano et al., 2009). Em termos práticos, o processo de aprendizagem de ontologia extrai conceitos e relações (padrões) de dados (semi-)estruturados ou de documentos sem estrutura, como textos (Cimiano et al., 2009).

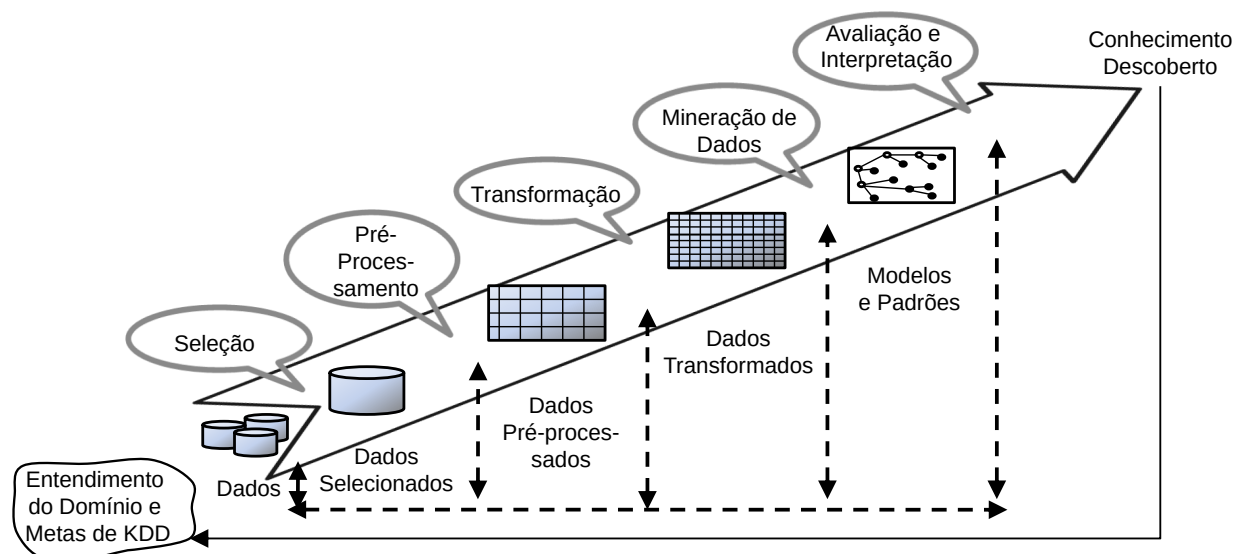


Figura 5.2: Processo de descoberta de conhecimento (Maimon e Rokach, 2010)

Utilizando tais técnicas para se extrair informação estruturada do pacote de laboratório não padronizado ao longo do processo de KDD, a primeira atividade do *workflow* de empacotamento é trivial: a seleção das informações correspondentes a cada fase do processo de experimentação.

Após realizar a atividade de *matching* dos esquemas, instancia-se as informações que apresentam os respectivos conceitos definidos na ontologia. As informações que restarem dessa atividade, sem conceito correspondente, indicam quais conceitos são faltantes, o que provê recursos para realizar a evolução da ontologia (Kondylakis e Plexousakis, 2010). Evolução de ontologia é a sua adapta-

ção oportuna para condizer com mudanças nas regras de negócio ou com tendências nas instâncias (Stojanovic et al., 2002).

Por fim, além de operacionalizar o *workflow* com a implementação de todas as suas atividades propostas, é preciso executá-lo de modo iterativo, realizando o empacotamento de estudos publicados na literatura. Dessa maneira, atinge-se o objetivo principal deste trabalho que consiste em evoluir a *EXPEROntology*, a fim de que se torne um padrão semântico dos conjuntos de informação que um pacote de laboratório pode conter. Para isso, deve-se adotar o mesmo procedimento da validação da ferramenta PontoLab. Após uma ampla revisão, de acordo com a abordagem proposta por Cruzes et al. (2007a), deve-se extrair as informações de publicações de experimentos.

5.4 Atividades e Cronograma

Este projeto de mestrado foi dividido nas atividades descritas a seguir:

1. Créditos em Disciplinas: no primeiro ano foram obtidos os créditos necessários, de acordo com as normas vigentes no programa;
2. Revisão Bibliográfica: está sendo feito um levantamento bibliográfico para fundamentar o desenvolvimento deste trabalho. Parte dos resultados dessa fase estão registrados neste monografia, que é a base para o Exame de Qualificação;
3. Exame de Qualificação;
4. Implementação do *workflow* e sua validação (primeira validação com dados de experimentos disponíveis na literatura);
5. Instanciação de Pacotes de Laboratório usando o *workflow* desenvolvido com o objetivo de padronizá-los;
6. Dissertação e Defesa: os resultados obtidos, juntamente com a descrição detalhada da implementação, serão apresentados na dissertação.

Considerando que o primeiro trimestre iniciou-se no mês de março de 2011, tais atividades estão organizadas de acordo com o cronograma apresentado na Tabela 5.1, que indica o progresso das atividades já concluídas.

Tabela 5.1: Cronograma de atividades do projeto de mestrado

Atividades	Trimestres							
	1	2	3	4	5	6	7	8
Créditos em Disciplinas	✓	✓	✓	✓				
Revisão Bibliográfica	✓	✓	✓	✓	✓	*		
Exame de Qualificação						*		
Implementação da Workflow						*	*	
Instanciação de Pacotes de Laboratório							*	
Dissertação e Defesa							*	*

5.5 Resultados Parciais

Para estabelecer um modelo que seja um padrão, é preciso instanciar pacotes de laboratório com diferentes estruturas de informação, a fim de que a *EXPEROntology* consiga absorver diferentes modelos de dados. Assim, durante a execução iterativa do *workflow*, seria ideal instanciar pacotes de laboratório provenientes de diferentes grupos de pesquisa.

[Protocols in the use of ESE artifacts \(Basili et al., 2007\)](#)

Entretanto, isso demandaria uma complexa coordenação de iniciativas entre os pesquisadores, pois há dificuldades para acessar os dados gerados pelos experimentos. Em termos práticos, não há livre acesso aos bancos de dados, documentos XML ou planilhas dos diferentes grupos de pesquisa em ES. Uma situação que ilustra essa dificuldade é a extração de evidências a partir dos experimentos já realizados. Em vez de acessar diretamente os pacotes de laboratório, os pesquisadores realizam revisões sistemáticas de artigos publicados na literatura da área ([Cruzes et al., 2007b](#)).

Essa dificuldade em se acessar diretamente os pacotes de laboratório favorece o uso de artigos publicados pelos pesquisadores para acessar as informações sobre o experimento. Adicionalmente, a utilização dos artigos como entrada do *workflow* de empacotamento é uma abordagem que favorece a evolução da *EXPEROntology*, pois a prática de publicação de ES Experimental apresenta heterogeneidade nos conjuntos de informações que descrevem cada experimento ([Jedlitschka et al. \(2008\)](#); [Carver \(2010\)](#)). Cada artigo pode apresentar um conjunto de informações distinto, o que implica em um modelo subjacente de pacote de laboratório distinto.

Dessa maneira, o processo de *matching* recebe como entradas a *EXPEROntology* e um documento PDF com a descrição das informações de um experimento. O documento (que representa o pacote de laboratório) não se trata de um esquema de dados, como requerido pelo processo. Entretanto, as informações contidas no texto estão de acordo com um modelo de dados subjacente, ou seja, os elementos de dado estão presentes no documento. Apesar de não estarem estruturados, estão dispersos em meio ao texto em linguagem natural.

De acordo com a Figura ??, as entradas do processo de *matching* devem ser dois esquemas. No caso do *workflow* de empacotamento, um dos esquemas é a *EXPEROntology* e o outro é o esquema subjacente do conjunto de informações representado no documento PDF do artigo. A fase de execução do *matcher* precisa comparar os elementos de ambos os esquemas de entrada. Entretanto, no caso do documento PDF, não há elementos para serem utilizados na comparação. As informações que devem ser separadas em elementos de dados estão espalhadas ao longo do texto.

Assim, conforme citado na Seção 5.3, o processo de KDD se torna o responsável por descobrir os elementos de informação em meio ao texto em linguagem natural. A partir de um documento PDF (um formato de entrada não estruturado), o processo de KDD deve devolver como saída um formato estruturado de pacote de laboratório (que ainda não está padronizado). Com os dados estruturados a partir do documento PDF, é possível iniciar a fase de execução do *matcher*, conforme está esquematizado na Figura 5.3.

O processo de KDD, como apresentado na Figura 5.2, recebe como entrada uma base de dados, e, a partir desses dados estruturados descobre algum conhecimento novo, que não estava explicitamente declarado. No caso do pré-processamento de pacotes de laboratório, o processo de KDD vai receber

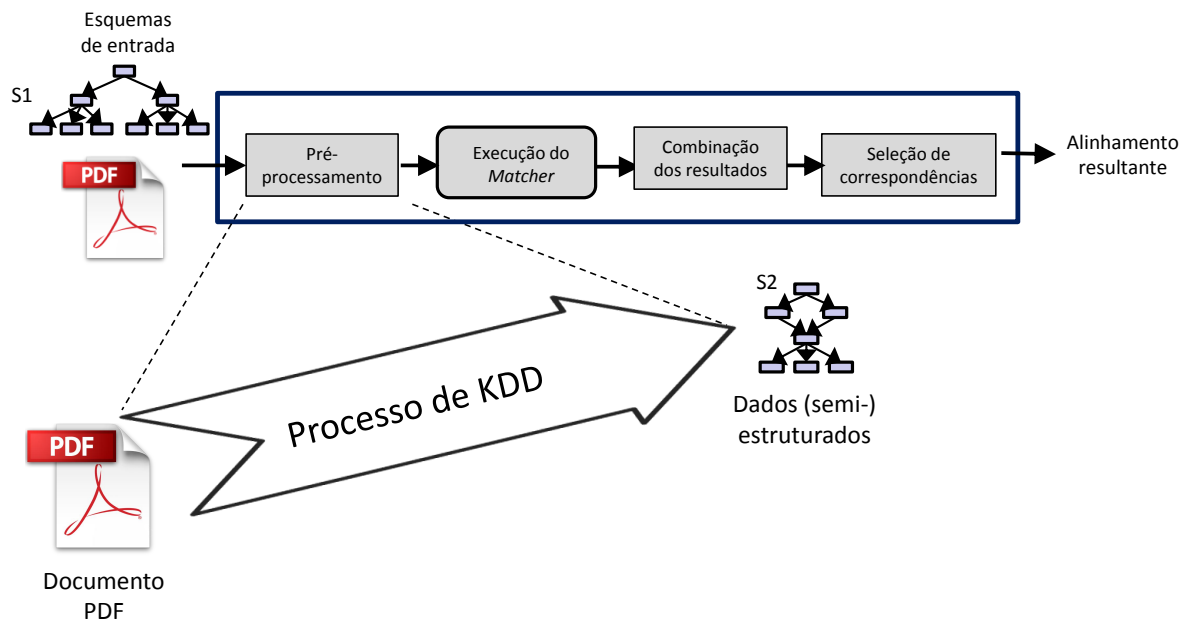


Figura 5.3: Etapa de pré-processamento do processo de *matching*

como entrada um documento em linguagem natural e extrair elementos de dados de modo a compor um pacote de laboratório estruturado. Contextualizando na implementação do processo de *matching* (vide Figura ??), a fase de pré-processamento pode ser considerada uma instância de processo de KDD adaptada, como ilustrado na Figura 5.4.

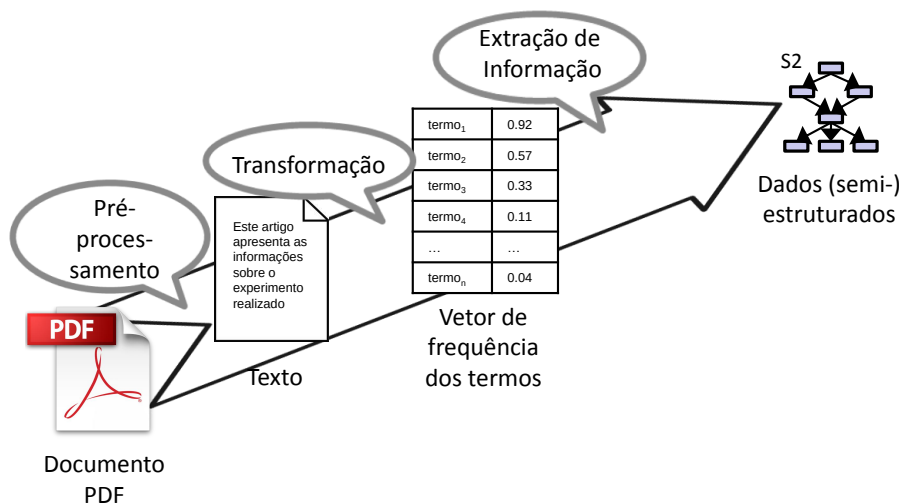


Figura 5.4: Processo de KDD adaptado

O formato estruturado gerado (saída do processo de KDD adaptado) deve prover os elementos de informação a serem comparados com os conceitos da *EXPEROntology* durante a atividade de *matching* do *workflow*. Como se trata de artigos que descrevem o experimento em linguagem natural, esses elementos devem ser formados por termos do texto que os representem. Conforme ilustrado na Figura 5.5, esses termos representativos devem ser extraídos a partir do texto do arquivo PDF. Por isso, a primeira etapa do processo de KDD consiste na extração de texto.

O documento PDF embute muitos tipos de dados (textos, imagens, formulários, textos, fontes),

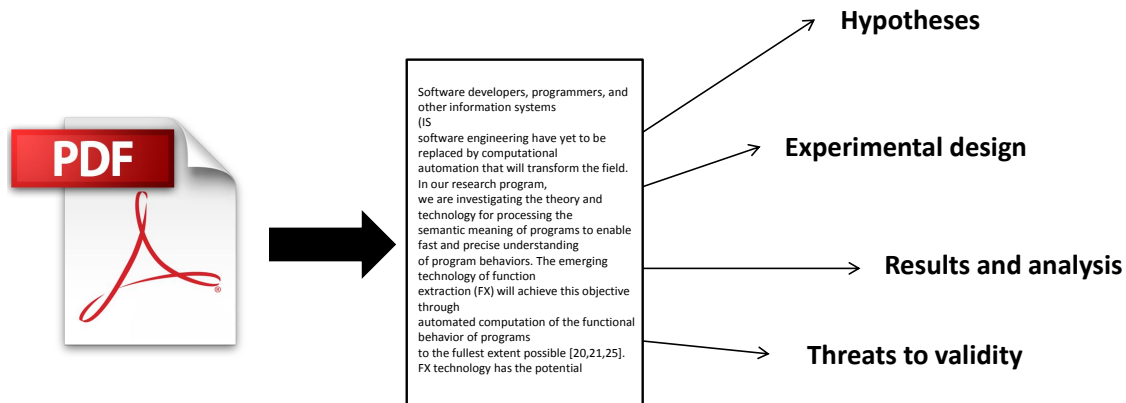


Figura 5.5: Extração de elementos do arquivo PDF

Tabela 5.2: Bibliotecas e ferramentas de extração de texto de arquivos PDF (Pitale e Sharma, 2011)

Ferramenta	Imagem	Texto	Fonte	Documentação disponível
SciPlore Xtract		✓		
PDFlibTET	✓	✓	✓	✓
Greenstone		✓		
3-Height PDF Extract	✓	✓	✓	✓
Solid Converter		✓	✓	✓
jPadel	✓	✓	✓	✓
pdftohtml	✓	✓	✓	
PDFBox	✓	✓		✓
ICEpdf	✓	✓	✓	✓
iText	✓	✓	✓	✓

além de restringir a edição direta do conteúdo. Essas características contribuem para a segurança do documento, mas por outro lado, também dificultam a extração direta de texto (Pitale e Sharma, 2011). Não obstante, algumas ferramentas e bibliotecas implementam a extração de conteúdo de arquivos PDF. Na Tabela 5.2 está especificada, para algumas delas, a capacidade de extrair os conteúdos do arquivo além de indicar se há documentação disponível. Para determinar qual deve ser usada para implementar a extração de texto do documento PDF, foi feito um comparativo entre essas opções.

Desconsiderando o que é *software* comercial e que não apresenta documentação disponível, restam as opções PDFBox, ICEpdf, iText. ICEpdf¹ é uma ferramenta para visualizar e manipular documentos PDF, em que a extração de texto é uma das funcionalidades disponíveis, no esquema copiar e colar. Sendo assim, não é adequada para os propósitos deste trabalho, uma vez que é preciso extrair o texto automaticamente de muitos artigos durante a instanciação de pacotes de laboratório pelo *work-flow*. Já no caso da biblioteca iText², apesar de apresentar documentação de alta qualidade (Lowagie, 2011), o seu foco está em gerar documentos dinamicamente (por exemplo, a partir de arquivos XML ou banco de dados). Existe a funcionalidade de extrair texto, mas apresenta limitações quanto à codificação das fontes usadas no documento.

¹<http://www.icesoft.org/projects/ICEpdf>

²<http://itextpdf.com/>

Assim, a biblioteca Java PDFBox³ é a mais adequada para se utilizar na implementação do *work-flow*. Além da criação e manipulação de documentos, também permite a extração automática de texto do arquivo PDF. A biblioteca consegue lidar com diferentes codificações de fonte e, apesar de apresentar problemas na organização do texto gerado (Hasan et al., 2012), consegue seguir a sequência de colunas de texto do documento. Portanto, o texto extraído pode ser usado pelas etapas subsequentes sem problemas maiores.

Dessa maneira, a atividade de mineração de dados no processo de KDD deve ser substituída por extração de informação (Mooney e Bunescu, 2005).

Na Figura 5.5, pode-se ver que o objetivo de extrair o texto do arquivo PDF é obter os termos representativos dos elementos que devem compor o pacote de laboratório estruturado. Localizar trechos específicos de dados em documentos em linguagem natural a fim de extrair informação estruturada de texto não estruturado caracteriza uma atividade de Extração de Informação (Mooney e Bunescu, 2005).

³<http://pdfbox.apache.org>

Considerações finais

Referências Bibliográficas

ALEKSOVSKI, Z.; KLEIN, M.; TEN KATE, W.; VAN HARMELEN, F. Matching unstructured vocabularies using a background ontology. In: *Proc. 15th International Conference on Knowledge Engineering and Knowledge Management (EKAW)*, Praha (CZ), 2006, p. 182–197 (*Lecture notes in computer science*, v.4248).

AMARAL, E. A. G. G.; TRAVASSOS, G. H. A package model for software engineering experiments. In: *Proceedings of ISESE 2003 - International Symposium on Empirical Software Engineering*, 2003, p. 21–22.

BAADER, F.; HORROCKS, I.; SATTTLER, U. *Handbook on ontologies*, cap. Description Logics Springer Verlag, p. 01–17, 2009.

BARBOSA, E. F.; NAKAGAWA, E. Y.; MALDONADO, J. C. Towards the establishment of an ontology of software testing. In: *SEKE (Software Engineering & Knowledge Engineering)*, 2006, p. 522–525.

BASILI, V. R.; SELBY, R. W. Comparing the effectiveness of software testing strategies. *IEEE Transaction on Software Engineering*, v. 13, n. 12, p. 1278–1296, 1987.

BASILI, V. R.; SHULL, F.; LANUBILE, F. Building knowledge through families of experiments. *IEEE Transactions on Software Engineering*, v. 25, n. 4, p. 456–473, 1999.

BASILI, V. R.; ZELKOWITZ, M. V.; SJØBERG, D. I.; JOHNSON, P.; COWLING, A. J. Protocols in the use of Empirical Software Engineering artifacts. *Empirical Software Engineering*, v. 12, n. 1, p. 107–119, 2007.

BIOLCHINI, J.; MIAN, P. G.; NATALI, A. C. C.; CONTE, T. U.; TRAVASSOS, G. H. Scientific research ontology to support systematic review in software engineering. *Advanced Engineering Informatics*, v. 21, n. 2, p. 133–151, 2007.

BOUQUET, P.; EHRIG, M.; EUZENAT, J.; FRANCONI, E.; HITZLER, P.; KRÖTZSCH, M.; SERAFINI, L.; STAMOU, G.; SURE, Y.; TESSARIS, S. *Specification of a common framework for characterizing alignment*. Deliverable D2.2.1, Knowledge web NoE, 2004.

- BROOKS, A.; ROPER, M.; WOOD, M.; DALY, J.; MILLER, J. *Guide to advanced empirical software engineering*, cap. Replication's Role in Software Engineering Springer-Verlag London, p. 365–379, 2008.
- CARVER, J. C. Towards reporting guidelines for experimental replications: A proposal. In: *International Workshop on Replication in Empirical Software Engineering Research (RESER)*, 2010.
- CIMIANO, P.; MAEDCHE, A.; STAAB, S.; VÖLKER, J. *Handbook on ontologies*, cap. Ontology Learning Springer-Verlag, p. 1–17, 2009.
- COLLINS, R. W.; HEVNER, A. R.; WALTON, G. H.; LINGER, R. C. The impacts of function extraction technology on program comprehension: A controlled experiment. *Information and Software Technology*, v. 50, n. 11, p. 1165–1179, 2008.
- CRUZES, D.; MENDONÇA, M.; BASILI, V.; SHULL, F.; JINO, M. Extracting information from experimental software engineering papers. In: *Proceedings of the XXVI International Conference of the Chilean Society of Computer Science*, Washington, DC, USA: IEEE Computer Society, 2007a, p. 105–114.
- CRUZES, D.; MENDONÇA, M.; BASILI, V.; SHULL, F.; JINO, M. Automated information extraction from empirical software engineering literature: Is that possible? In: *Proceedings of the First International Symposium on Empirical Software Engineering and Measurement*, Washington, DC, USA: IEEE Computer Society, 2007b, p. 491–493.
- EUZENAT, J.; SHVAIKO, P. *Ontology matching*. Springer-Verlag, 2007.
- FALBO, R.; RUY, F. B.; MORO, R. D. Using ontologies to add semantics to a software engineering environment. In: *SEKE (Software Engineering & Knowledge Engineering)*, 2005, p. 151–156.
- FAYYAD, U. M.; PIATETSKY-SHAPIO, G.; SMYTH, P. From data mining to knowledge discovery in databases. *AI Magazine*, v. 17, n. 3, p. 37–54, 1996a.
- FAYYAD, U. M.; PIATETSKY-SHAPIO, G.; SMYTH, P. Knowledge discovery and data mining: Towards a unifying framework. In: *Knowledge Discovery and Data Mining*, 1996b, p. 82–88.
- FELLBAUM, C. *WordNet: an electronic lexical database*. Cambridge (MA US): The MIT Press, 1998.
- GARCIA, R. E. *Vidaese: processo de visualização exploratória para apoio a estudos experimentais em verificação, validação e teste de software*. Tese de Doutorado, Instituto de Ciências Matemáticas e de Computação (ICMC-USP), 2006.

- GARCIA, R. E.; HOHN, E. N.; BARBOSA, E. F.; MALDONADO, J. C. An ontology for controlled experiments on software engineering. In: *SEKE (Software Engineering & Knowledge Engineering)*, Knowledge Systems Institute Graduate School, 2008, p. 685–690.
- HAKIMPOUR, F.; GEPPERT, A. Resolving semantic heterogeneity in schema integration: an ontology-based approach. In: *Proceedings of the International Conference on Formal Ontology in Information Systems*, New York, NY, USA: ACM, 2001, p. 297–308.
- HALEVY, A. Why your data won't mix: Semantic heterogeneity. *ACM Queue*, v. 3, p. 50–58, 2005.
- HASAN, I.; PARAPAR, J.; BARREIRO, A. Improving the extraction of text in pdfs by simulating the human reading order. *J. of UCS*, v. 18, n. 5, p. 623–649, 2012.
- JEDLITSCHKA, A.; CIOLKOWSKI, M.; PFAHL, D. *Reporting guidelines for controlled experiments in software engineering*. Relatório Técnico, Fraunhofer Institute for Experimental Software Engineering, Germany, 2007.
- JEDLITSCHKA, A.; CIOLKOWSKI, M.; PFAHL, D. *Guide to advanced empirical software engineering*, cap. Reporting Experiments in Software Engineering London: Springer-Verlag, p. 201–228, 2008.
- JURISTO, N.; MORENO, A. M. *Basics of software engineering experimentation*. Kluwer, 2001.
- KITCHENHAM, B. *Procedures for performing systematic reviews*. Relatório Técnico, Keele University Joint Technical Report TR/SE-0401, 2004.
- KITCHENHAM, B. A.; PFLEEGER, S. L.; PICKARD, L. M.; JONES, P. W.; HOAGLIN, D. C.; EMAM, K. E.; ROSENBERG, J. Preliminary guidelines for empirical research in software engineering. *IEEE Transactions on Software Engineering*, v. 28, n. 8, p. 721–734, 2002.
- KITCHENHAM, B. A.; TRAVASSOS, G. H.; VON MAYRHAUSER, A.; NIESSINK, F.; SCHNEIDEWIND, N. F.; SINGER, J.; TAKADA, S.; VEHVILAINEN, R.; YANG, H. Towards an ontology of software maintenance. *Journal of Software Maintenance*, v. 11, n. 6, p. 365–389, 1999.
- KONDYLAKIS, H.; PLEXOUSAKIS, D. Enabling ontology evolution in data integration. In: *Proceedings of the 2010 EDBT/ICDT Workshops*, New York, NY, USA: ACM, 2010, p. 38:1–38:7.
- LOWAGIE, B. *iText in action: Creating and manipulating PDF*. 2nd ed. Greenwich, CT, USA: Manning Publications Co., 2011.
- LUCIA, A. D.; GRAVINO, C.; OLIVETO, R.; TORTORA, G. An experimental comparison of er and uml class diagrams for data modelling. *Empirical Software Engineering*, v. 15, n. 5, p. 455–492, 2010.

- MAEDCHE, A.; STAAB, S. Ontology learning. In: *Handbook on Ontologies*, Springer Verlag, p. 173–190, 2004.
- MAIMON, O.; ROKACH, L. *Data mining and knowledge discovery handbook*, capítulo. Introduction to Knowledge Discovery and Data Mining Springer, p. 01–15, 2010.
- MANNING, C. D.; RAGHAVAN, P.; SCHATZ, H. *Introduction to information retrieval*, capítulo. Scoring, term weighting and the vector space model New York, NY, USA: Cambridge University Press, p. 109–133, 2008.
- MENDONÇA, M. G.; MALDONADO, J. C.; DE OLIVEIRA, M. C. F.; CARVER, J.; FABBRI, S. C. P. F.; SHULL, F.; TRAVASSOS, G. H.; HOHN, E. N.; BASILI, V. R. A framework for software engineering experimental replications. In: *ICECCS*, 2008, p. 203–212.
- MILLER, G. WordNet: A lexical database for english. *Communications of the ACM*, v. 38, n. 11, p. 39–41, 1995.
- MOONEY, R. J.; BUNESCU, R. Mining knowledge from text using information extraction. *ACM SIGKDD Explorations Newsletter*, v. 7, n. 1, p. 3–10, 2005.
- NOY, N. F. *Handbook on ontologies*, capítulo. Ontology Mapping Springer Verlag, p. 573–589, 2009.
- NOY, N. F.; MCGUINNESS, D. L. *Ontology development 101: A guide to creating your first ontology*. Relatório Técnico, Stanford University, 2001.
- PITALE, S.; SHARMA, T. Information extraction for Portable Document Format. *International Journal of Computer Technology and Applications*, v. 2, p. 2047–2051, 2011.
- RAHM, E. *Schema matching and mapping*, capítulo. Towards Large-Scale Schema and Ontology Matching Springer Publishing Company, Incorporated, p. 3–27, 2011.
- RAHM, E.; BERNSTEIN, P. A survey of approaches to automatic schema matching. *The VLDB Journal*, v. 10, n. 4, p. 334–350, 2001.
- ROBERTSON, S.; SPÄRCK JONES, K. Relevance weighting of search terms. *Journal of the American Society for Information Science*, v. 27, n. 3, p. 129–146, 1976.
- SALTON, G. *The SMART retrieval system: experiments in automatic information processing*. Englewood Cliffs (NJ US): Prentice Hall, 1971.
- SALTON, G.; MCGILL, M. *Introduction to modern information retrieval*. New York (NY US): McGraw-Hill, 1983.
- SCATALON, L. P.; GARCIA, R. E.; CORREIA, R. C. M. Packaging controlled experiments using an evolutionary approach based on ontology. In: *International Conference on Software Engineering and Knowledge Engineering (SEKE 2011)*, 2011a, p. 408–413.

- SCATALON, L. P.; GARCIA, R. E.; CORREIA, R. C. M. Uma ferramenta para instanciação de ontologia de experimentos controlados em engenharia de software. In: *6ª Conferência Ibérica de Sistemas e Tecnologias de Informação (CISTI 2011)*, 2011b.
- SHULL, F.; BASILI, V. R.; CARVER, J.; MALDONADO, J. C.; TRAVASSOS, G. H.; MENDONÇA, M. G.; FABBRI, S. C. P. F. Replicating software engineering experiments: Addressing the tacit knowledge problem. 2002.
- SHULL, F.; CARVER, J.; TRAVASSOS, G. H.; MALDONADO, J. C.; CONRADI, R.; BASILI, V. R. Replicated studies: building a body of knowledge about software reading techniques. In: *Lecture notes on empirical software engineering*, River Edge, NJ, USA: World Scientific Publishing Co., Inc., p. 39–84, 2003.
- SHVAIKO, P.; EUZENAT, J. A survey of schema-based matching approaches. *Journal on Data Semantics*, v. IV, p. 146–171, 2005.
- SHVAIKO, P.; EUZENAT, J. Ontology matching: state of the art and future challenges. Disponível em <http://ontologymatching.org/publications.html>, 2012.
- SINGER, J. Using the APA style guidelines to report experimental results. In: *Proceedings of Workshop on Empirical Studies in Software Maintenance*, 1999, p. 71–75.
- STOJANOVIC, L.; STOJANOVIC, N.; HANDSCHUH, S. Evolution of the metadata in the ontology-based knowledge management systems. In: *Proceedings of the 1st German Workshop on Experience Management: Sharing Experiences about the Sharing of Experience*, 2002, p. 65–77.
- TRAVASSOS, G. H.; GUROV, D.; AMARAL, E. A. G. *Introdução à engenharia de software experimental*. Relatório Técnico, COPPE/UFRJ, 2002.
- WOHLIN, C.; RUNESON, P.; HÖST, M.; OHLSSON, M.; REGNELL, B.; WESSLÉN, A. *Experimentation in software engineering: An introduction*. Boston, USA: Kluwer Academic Publishers, 2000.
- XIE, S.; KRAEMER, E.; STIREWALT, R. E. K.; DILLON, L. K.; FLEMING, S. D. Assessing the benefits of synchronization-adorned sequence diagrams: two controlled experiments. In: *Proceedings of the 4th ACM Symposium on Software visualization*, New York, NY, USA: ACM, 2008, p. 9–18.